



Joy World Cityfuns Vietnam Co. Ltd

Software Architecture Document

for

JPULSE

Phiên bản 1.0

Tài liệu thuộc sở hữu của **CÔNG TY TNHH JOY WORLD CITYFUNS VIETNAM**

Mục lục

Chương 1: Giới thiệu	1
1.1. Mục đích tài liệu	1
1.2. Phạm vi hệ thống	1
1.3. Đối tượng đọc tài liệu	2
1.4. Thuật ngữ và chữ viết tắt	2
1.5. Tài liệu tham chiếu	4
Chương 2: Bối cảnh và yêu cầu kiến trúc.....	5
2.1. Bối cảnh nghiệp vụ và mục tiêu hệ thống.....	5
2.2. Các bên liên quan và mối quan tâm của họ	5
2.3. Tác nhân và hệ thống bên ngoài	6
2.4. Ranh giới hệ thống.....	6
2.5. Các chức năng ảnh hưởng đến kiến trúc.....	7
2.6. Yêu cầu chất lượng	8
2.6.1. Bảo mật và quyền riêng tư.....	8
2.6.2. Hiệu năng.....	8
2.6.3. Khả năng mở rộng	9
2.6.4. Tính sẵn sàng và khả năng phục hồi	9
2.6.5. Khả năng bảo trì.....	9
2.7. Ràng buộc và giả định thiết kế	10
Chương 3: Tổng quan và chiến lược kiến trúc	11
3.1. Nguyên tắc thiết kế	11
3.2. Kiểu kiến trúc được lựa chọn.....	12
3.3. Sơ đồ kiến trúc tổng thể.....	13

3.4. Các thành phần chính và trách nhiệm.....	14
3.4.1. Giao diện web JPULSE	14
3.4.2. API nghiệp vụ JPULSE	15
3.4.3. Gói kiểu dữ liệu và hợp đồng dùng chung.....	16
3.4.4. Kho dữ liệu và lưu trữ tệp.....	17
3.4.5. Dịch vụ tích hợp bên ngoài.....	18
3.5. Cách các thành phần giao tiếp	19
3.6. Lý do chọn giải pháp kiến trúc	21
Chương 4: Các góc nhìn kiến trúc	23
4.1. Góc nhìn ngữ cảnh – C4 System Context.....	23
4.2. Góc nhìn ứng dụng và kho dữ liệu – C4 Container	24
4.3. Góc nhìn thành phần – C4 Component.....	26
4.3.1. Thành phần giao diện web	26
4.3.2. Thành phần API nghiệp vụ	27
4.4. Góc nhìn động.....	27
4.4.1. Luồng xác thực và phân quyền	27
4.4.2. Luồng nhập xuất và điều chuyển kho	29
4.4.3. Luồng tích hợp JPOS và hóa đơn điện tử.....	32
4.5. Góc nhìn triển khai	36
Chương 5: Dữ liệu, giao tiếp và luồng xử lý	37
5.1. Tổng quan dữ liệu của hệ thống	37
5.2. Cấu trúc dữ liệu và nơi lưu trữ.....	37
5.2.1. Dữ liệu nghiệp vụ và phân quyền	37
5.2.2. Tệp và tài liệu	37

5.2.3. Dữ liệu tích hợp và nhật ký hệ thống.....	37
5.3. Luồng dữ liệu giữa các thành phần.....	37
5.4. Giao diện kết nối và API chính.....	37
5.4.1. API giữa giao diện web và máy chủ	37
5.4.2. Kết nối với dịch vụ bên ngoài.....	37
5.5. Xác thực, phân quyền và bảo vệ dữ liệu.....	37
5.6. Các luồng xử lý chính.....	37
5.6.1. Xác thực và truy cập theo vai trò, cơ sở	37
5.6.2. Quản lý nhập xuất, điều chuyển và kiểm kê.....	37
5.6.3. Ghi nhận doanh thu và phát hành hóa đơn	37
5.6.4. Chấm công và nghỉ phép	37
5.7. Xử lý lỗi và tính nhất quán dữ liệu	37
Chương 6: Triển khai và vận hành	38
6.1. Các môi trường triển khai.....	38
6.1.1. Môi trường phát triển.....	38
6.1.2. Môi trường kiểm thử.....	38
6.1.3. Môi trường vận hành	38
6.2. Sơ đồ triển khai.....	38
6.3. Cấu hình ứng dụng và quản lý thông tin bí mật	38
6.4. Quy trình triển khai và cập nhật	38
6.5. Ghi log và giám sát	38
6.6. Sao lưu và khôi phục dữ liệu	38
6.7. Khả năng mở rộng và giới hạn vận hành.....	38
Chương 7: Quyết định kiến trúc và đánh giá.....	39

7.1. Danh sách quyết định kiến trúc quan trọng	39
7.2. Lý do lựa chọn công nghệ và phương án thiết kế.....	39
7.3. Các phương án đã cân nhắc	39
7.4. Đánh đổi của phương án được chọn	39
7.5. Đánh giá kiến trúc theo yêu cầu chất lượng	39
7.6. Rủi ro kỹ thuật và hướng xử lý.....	39
Chương 8: Kết luận và phụ lục	40
8.1. Tóm tắt kiến trúc hệ thống.....	40
8.2. Những giới hạn hiện tại	40
8.3. Hướng phát triển kiến trúc	40
8.4. Phụ lục sơ đồ.....	40
8.5. Phụ lục bảng mô tả thành phần, dữ liệu hoặc API.....	40
8.6. Danh mục thuật ngữ.....	40

Chương 1: Giới thiệu

1.1. Mục đích tài liệu

Tài liệu kiến trúc phần mềm này mô tả cách tổ chức hệ thống JPULSE để đáp ứng phạm vi và các yêu cầu đã xác định trong tài liệu SRS-JPULSE. Tài liệu xác định ranh giới hệ thống, trách nhiệm của các thành phần, cách trao đổi và lưu trữ dữ liệu, các luồng xử lý quan trọng và mô hình triển khai.

Tài liệu là cơ sở cho nhóm thiết kế và phát triển triển khai nhất quán giữa giao diện web, API, dữ liệu và các tích hợp; đồng thời hỗ trợ nhóm kiểm thử, vận hành và bảo trì đánh giá tác động khi hệ thống thay đổi. Yêu cầu nghiệp vụ chi tiết, quy tắc nghiệp vụ và tiêu chí chấp nhận tiếp tục được quản lý trong SRS; các chương sau của tài liệu này tập trung giải thích các quyết định kiến trúc tương ứng.

1.2. Phạm vi hệ thống

JPULSE là nền tảng quản trị vận hành của Joy World, bao gồm quản lý kho và tồn kho, nhân sự và chăm công, doanh thu và chi phí nội bộ, hóa đơn điện tử, quản trị truy cập, thiết bị POS và chiến dịch voucher. Phạm vi kiến trúc của tài liệu bao trùm các thành phần phần mềm phục vụ những nhóm chức năng đó cùng các quan hệ giữa chúng.

Các thành phần được xem xét gồm giao diện web dùng trên máy tính và thiết bị di động, API xử lý nghiệp vụ, hợp đồng dữ liệu dùng chung, dịch vụ xác thực, kho dữ liệu và lưu trữ tệp. Tài liệu cũng xét các điểm kết nối với JPOS, hệ thống ERP cũ, MISA meInvoice và dịch vụ email Brevo; Firebase và Google Cloud là các dịch vụ nền tảng được SRS xác định cho dữ liệu, tệp và môi trường vận hành.

Ranh giới của tài liệu là kiến trúc hệ thống JPULSE và các giao diện tích hợp tại phía JPULSE. Đặc tả từng màn hình, use case, trường dữ liệu, endpoint và quy tắc nghiệp vụ chi tiết thuộc SRS hoặc tài liệu chuyên biệt. Theo phạm vi SRS, tính lương, kế toán tổng hợp và cổng thương mại điện tử cho khách hàng không nằm trong phạm vi chức năng của JPULSE.

1.3. Đối tượng đọc tài liệu

Tài liệu dành cho nhóm thiết kế và phát triển, nhóm kiểm thử, người quản trị và vận hành hệ thống, cùng các bên phụ trách nghiệp vụ cần hiểu ranh giới và tác động của thay đổi. Mỗi nhóm sử dụng tài liệu theo mỗi quan tâm sau:

Nhóm thiết kế và phát triển: dùng để xác định trách nhiệm của giao diện, API, dữ liệu và tích hợp; làm căn cứ thiết kế chi tiết và sửa đổi hệ thống.

Nhóm kiểm thử: dùng để xây dựng các kịch bản kiểm thử luồng liên thành phần, phân quyền, tích hợp và xử lý lỗi dựa trên kiến trúc đã mô tả.

Người quản trị và vận hành: dùng để cấu hình môi trường, quản lý quyền truy cập, theo dõi sự cố, sao lưu và khôi phục dữ liệu.

Ban quản lý và các bộ phận nghiệp vụ: dùng để kiểm tra sự phù hợp giữa giải pháp kỹ thuật với phạm vi, yêu cầu chất lượng và các phụ thuộc nghiệp vụ trong SRS.

1.4. Thuật ngữ và chữ viết tắt

Các thuật ngữ dưới đây được dùng thống nhất trong tài liệu; tên dịch vụ và nhóm chức năng tuân theo cách gọi trong SRS-JPULSE.

Bảng 1-1: Thuật ngữ và chữ viết tắt

Thuật ngữ	Tên đầy đủ	Ý nghĩa	Ghi chú
J-PULSE / JPULSE	Platform for Unified Leadership, Service & Enterprise of Joy World	Hệ thống quản trị các nghiệp vụ vận hành được mô tả trong SRS.	
SRS	Software Requirements Specification	Tài liệu đặc tả yêu cầu phần mềm của JPULSE.	

Thuật ngữ	Tên đầy đủ	Ý nghĩa	Ghi chú
SAD	Software Architecture Document	Tài liệu mô tả kiến trúc phần mềm của JPULSE.	
WMS	Warehouse Management System	Nhóm chức năng quản lý kho, tồn kho và luân chuyển hàng hóa.	
JPOS	Hệ thống bán hàng JPOS	Hệ thống POS/kênh bán hàng trao đổi dữ liệu với JPULSE.	
API	Application Programming Interface	Giao diện trao đổi dữ liệu giữa các thành phần hoặc hệ thống.	
RBAC	Role-Based Access Control	Cơ chế phân quyền theo vai trò người dùng.	
Office Scope	Phạm vi cơ sở	Giới hạn dữ liệu và thao tác theo cơ sở được phân công.	
MFA	Multi-Factor Authentication	Xác thực người dùng bằng nhiều yếu tố.	
BOM	Bill of Materials	Định mức vật tư gắn với sản phẩm.	
ATP	Available to Promise	Số lượng có thể đáp ứng theo chính sách tồn kho.	
GCP	Google Cloud Platform	Nền tảng đám mây được SRS nêu cho môi trường vận hành.	
Firestore	Cloud Firestore	Kho dữ liệu dùng trong hệ sinh thái Firebase.	

Thuật ngữ	Tên đầy đủ	Ý nghĩa	Ghi chú
meInvoice	MISA meInvoice	Dịch vụ phát hành và quản lý hóa đơn điện tử tích hợp với JPULSE.	

1.5. Tài liệu tham chiếu

Nguồn tham chiếu chính là tài liệu SRS-JPULSE.docx, phiên bản 1.0. Chương 1 của SRS xác định phạm vi và thuật ngữ; Chương 2 mô tả bối cảnh, các bên liên quan, môi trường vận hành và ràng buộc; Chương 5 trình bày dữ liệu; Chương 6 trình bày tích hợp; Chương 7 xác định yêu cầu phi chức năng.

Khi thông tin trong tài liệu kiến trúc cần đối chiếu về chức năng hoặc phạm vi nghiệp vụ, áp dụng SRS-JPULSE làm tài liệu yêu cầu gốc. Đặc tả chi tiết của từng API, mô hình dữ liệu và cấu hình triển khai được tham chiếu tại các chương tương ứng của tài liệu kiến trúc khi những phần đó được hoàn thiện.

Chương 2: Bối cảnh và yêu cầu kiến trúc

2.1. Bối cảnh nghiệp vụ và mục tiêu hệ thống

Joy World từng quản lý nhiều hoạt động cửa hàng bằng hồ sơ giấy, bảng tính và hệ thống có phạm vi hạn chế. Theo SRS, dữ liệu phân tán, đối soát thủ công và mức độ liên kết cao trong phần mềm cũ gây khó khăn cho việc kiểm soát, bảo trì và mở rộng.

JPULSE được hình thành từ JWC-ERPS để thay thế dần cách vận hành đó bằng một nền tảng quản trị có giao diện và dịch vụ xử lý nghiệp vụ tách biệt.

Mục tiêu của JPULSE là số hóa quy trình, tập trung dữ liệu, bảo toàn lịch sử xử lý và giúp các bộ phận nhìn thấy trạng thái nghiệp vụ theo quyền được cấp. Hệ thống khởi đầu từ quản lý kho, sau đó mở rộng sang nhân sự, chi phí, doanh thu, hóa đơn điện tử và các nghiệp vụ quản trị khác. Kết nối với JPOS giúp dữ liệu bán hàng đến hệ thống quản trị gần thời gian thực; kết nối MISA meInvoice giảm các bước tổng hợp và phát hành hóa đơn thủ công.

Những mục tiêu trên dẫn đến các nhu cầu kiến trúc chính: tách trách nhiệm giữa giao diện, API và dữ liệu; duy trì một hợp đồng trao đổi nhất quán; kiểm soát quyền theo vai trò và cơ sở; xử lý an toàn các tác vụ kéo dài hoặc kết quả tích hợp chưa xác định; và mở rộng theo số cơ sở, người dùng và giao dịch mà không làm mất tính đúng đắn của dữ liệu (SRS, Chương 2).

2.2. Các bên liên quan và mối quan tâm của họ

Ban Giám đốc và Quản lý Vận hành cần số liệu kịp thời, quy trình thống nhất và khả năng giám sát theo cơ sở. Vì vậy, kiến trúc phải cung cấp dữ liệu báo cáo có nguồn gốc rõ ràng, cập nhật có kiểm soát và chỉ hiển thị trong phạm vi được phép.

Kho vận, Kế toán, Nhân sự và Marketing là các đơn vị sở hữu quy tắc nghiệp vụ. Kho vận quan tâm đến số dư tồn kho và chứng từ có thể truy vết; Kế toán quan tâm đến đối soát doanh thu, chi phí và hóa đơn; Nhân sự cần bảo vệ hồ sơ, chấm công và nghỉ phép; Marketing cần kiểm soát vòng đời chiến dịch và mã voucher. Quản lý cơ sở, nhân

viên kho, cửa hàng và thu ngân cần thao tác rõ ràng trên thiết bị phù hợp, kể cả các luồng được phép hoạt động khi kết nối không ổn định.

Quản trị hệ thống và IT cần quản lý tài khoản, quyền, cấu hình, bí mật, giám sát và khôi phục. Kiểm soát nội bộ cần audit log và lịch sử phê duyệt để xác định ai thực hiện thay đổi, khi nào và trong phạm vi nào. Nhóm Product, BA, Dev và QA cần ranh giới thành phần, hợp đồng dữ liệu và tiêu chí chất lượng đủ rõ để phát triển, kiểm thử và đánh giá tác động thay đổi (SRS, Mục 2.3).

2.3. Tác nhân và hệ thống bên ngoài

Tác nhân sử dụng JPULSE gồm Ban Giám đốc, Quản lý Vận hành, các phòng ban Kho vận, Kế toán, Nhân sự, Marketing, quản lý cơ sở, nhân viên cửa hàng, thu ngân, quản trị viên và người được cấp quyền kiểm toán. Khách hàng chỉ tương tác trong phạm vi cung cấp thông tin và nhận hóa đơn hoặc thông báo liên quan đến giao dịch; họ không truy cập dữ liệu quản trị nội bộ.

Các hệ thống trao đổi dữ liệu với JPULSE gồm JPOS/JoyWorld và hệ thống ERP cũ cho dữ liệu bán hàng, người dùng hoặc tồn kho; MISA meInvoice cho kết quả phát hành hóa đơn; Brevo cho email giao dịch; VietQR cho tra cứu thông tin doanh nghiệp. Google Cloud và Firebase cung cấp môi trường chạy, xác thực, Firestore và Cloud Storage. Cơ quan quản lý nhà nước đặt ra yêu cầu tuân thủ, không phải một điểm kết nối kỹ thuật mặc định (SRS, Mục 2.3–2.4).

Mỗi kết nối phải có chủ thể sở hữu dữ liệu và trách nhiệm xử lý rõ ràng. Hệ thống nguồn giữ quyền đối với dữ liệu gốc do mình phát hành; JPULSE xác thực yêu cầu, chuẩn hóa định danh, lưu trạng thái xử lý và đối soát kết quả cần thiết cho quy trình nội bộ. Kết quả trả về chưa xác định từ hệ thống ngoài không được coi là thành công.

2.4. Ranh giới hệ thống

Trong ranh giới JPULSE có giao diện người dùng do dự án cung cấp, API và tác vụ nền, quy tắc nghiệp vụ, dữ liệu nội bộ, tệp chứng từ, báo cáo, nhật ký kiểm toán và lớp điều phối tích hợp. JPULSE chịu trách nhiệm kiểm tra định danh, quyền theo vai trò và

phạm vi cơ sở, hợp lệ hóa đầu vào, chuyển trạng thái quy trình, lưu thay đổi và cung cấp kết quả cho người dùng hoặc hệ thống được phép.

Người dùng và các hệ thống kể ở Mục 2.3 nằm ngoài ranh giới triển khai của JPULSE. Mọi trao đổi phải đi qua giao diện, API, webhook hoặc tác vụ tích hợp đã xác định; hệ thống ngoài không truy cập trực tiếp kho dữ liệu nội bộ. JPULSE không kiểm soát hoạt động bên trong, thời gian sẵn sàng hay dữ liệu có thẩm quyền của MISA, Brevo, VietQR và các hệ thống bán hàng.

Theo SRS, JPULSE không thực hiện tính lương, kế toán tổng hợp hoặc công thương mại điện tử cho khách hàng. Ranh giới này giúp các quyết định lưu trữ, phân quyền và tích hợp tập trung vào dữ liệu và quy trình mà JPULSE thực sự sở hữu (SRS, Mục 1.2 và 2.4).

2.5. Các chức năng ảnh hưởng đến kiến trúc

Nhiệm vụ kho tạo ra các thay đổi liên quan đồng thời đến phiếu, phê duyệt và nhiều bucket tồn kho. Kiến trúc phải bảo đảm tính nguyên tử, kiểm tra lượng khả dụng, ngăn thao tác vượt quyền và lưu vết đủ để đối soát khi kiểm kê hoặc điều chuyển. Dữ liệu nhập tại hiện trường có thể cần đồng bộ lại sau khi mất kết nối (SRS: BC01–BC05, BC14).

Các luồng hóa đơn, doanh thu và POS chịu ảnh hưởng của hệ thống ngoài và trạng thái giao dịch bất đồng bộ. JPULSE phải ngăn phát hành trùng hóa đơn, nhận diện trạng thái chưa xác định, lưu khóa chống trùng và hỗ trợ đối soát. Dữ liệu JPOS được tổng hợp theo nguồn, trạng thái thanh toán và phạm vi cơ sở; tác vụ phát hành hoặc đồng bộ lớn cần chạy nền và hiển thị tiến độ (SRS: BC06–BC07, P04).

Hồ sơ nhân viên, chấm công, nghỉ phép, chi phí và voucher dùng chính sách, phê duyệt, số dư hoặc feature flag khác nhau. Chúng cần một cơ chế nhất quán để kiểm tra quyền, trạng thái, thời điểm nghiệp vụ và lịch sử thay đổi, nhưng vẫn giữ quy tắc riêng ở từng miền nghiệp vụ. Các thao tác nhạy cảm như xử lý chênh lệch tồn kho hoặc phát hành hóa đơn cần xác thực tăng cường theo SRS.

Tất cả luồng dùng chung định danh người dùng, vai trò, phạm vi cơ sở, cấu hình tích hợp, thông báo và audit. Vì vậy, kiểm soát truy cập và hợp đồng dữ liệu giữa giao diện, API, tác vụ nền và hệ thống ngoài là các mối quan tâm xuyên suốt, không thể chỉ kiểm tra tại màn hình người dùng.

2.6. Yêu cầu chất lượng

Các mục dưới đây tóm tắt những yêu cầu có tác động trực tiếp đến kiến trúc. Giá trị mục tiêu và tiêu chí nghiệm thu chi tiết thuộc Chương 7 của SRS; khi triển khai cần đo và đối chiếu theo môi trường tương ứng.

2.6.1. Bảo mật và quyền riêng tư

Người dùng web xác thực qua Firebase và phiên đăng nhập an toàn; API phải kiểm tra quyền theo vai trò và phạm vi cơ sở trước khi truy cập hoặc thay đổi dữ liệu. Khi ngữ cảnh quyền không tồn tại hoặc không hợp lệ, hệ thống từ chối mặc định. Firestore Rules áp dụng deny-by-default; các thay đổi nghiệp vụ quan trọng đi qua backend để kiểm tra trạng thái và ghi audit (SRS: SEC01–SEC03, TC06–TC07).

Bí mật tích hợp, khóa mã hóa và thông tin service account chỉ được cấp cho môi trường máy chủ; kết nối production dùng HTTPS/TLS, CORS theo danh sách cho phép và giới hạn tốc độ truy cập. Dữ liệu cá nhân, thông tin nhân sự, OTP, token và nội dung hóa đơn phải được tối thiểu hóa trong trao đổi, giới hạn quyền đọc và không ghi nguyên văn vào log. Thao tác rủi ro cao dùng OTP/MFA theo chính sách (SRS: SEC04–SEC10).

2.6.2. Hiệu năng

SRS đặt mục tiêu 95% yêu cầu API thông thường không gọi hệ thống ngoài hoàn tất trong 2 giây và 99% trong 5 giây ở tải danh định; màn hình nghiệp vụ đầu tiên có thể thao tác trong 3 giây trên mạng văn phòng ổn định. Danh sách dữ liệu dài cần phân trang hoặc tải tăng dần để giới hạn lượng dữ liệu truyền và xử lý trên client (SRS: P01–P02).

Lệnh phát hành hóa đơn hàng loạt, xuất voucher hoặc đồng bộ lớn cần được tiếp nhận nhanh, trả mã công việc và xử lý nền thay vì giữ kết nối chờ. Mọi lời gọi hệ thống

ngoài phải có timeout; SRS nêu ngưỡng 15 giây cho MISA và yêu cầu phân biệt lỗi có thể thử lại với trạng thái chưa xác định (SRS: P03–P04).

2.6.3. Khả năng mở rộng

Backend phải không phụ thuộc trạng thái chỉ nằm trong bộ nhớ hoặc ổ đĩa cục bộ để có thể mở rộng ngang. SRS ghi nhận cấu hình Cloud Run hiện tại tại asia-southeast1 với min 0, max 3 instance và concurrency 80; việc tăng giới hạn phải qua kiểm thử tải mà không cần thay đổi tính đúng đắn của nghiệp vụ (SRS: SCA01–SCA02, TC04).

Truy vấn và báo cáo phải giới hạn theo cơ sở, thời gian và số bản ghi, dùng index cùng cơ chế phân trang phù hợp Firestore. Các tác vụ fan-out đặt giới hạn song song; tác vụ nền dùng khóa chống trùng, lease và retry để nhiều instance không xử lý cùng một đơn vị công việc. Hợp đồng dữ liệu dùng chung cần có phương án chuyển tiếp khi thay đổi không tương thích (SRS: SCA03–SCA06).

2.6.4. Tính sẵn sàng và khả năng phục hồi

SRS đặt mục tiêu mức sẵn sàng tối thiểu 99,5% theo tháng cho API lõi. Kiến trúc cần health check, trạng thái công việc bền vững và cơ chế suy giảm có kiểm soát khi dịch vụ tích hợp không sẵn sàng; các nghiệp vụ độc lập vẫn phải hoạt động. Cập nhật tồn kho, số dư nghỉ phép và khóa công việc cần transaction hoặc batch; yêu cầu có thể gửi lại phải có định danh ổn định để không nhân đôi tác động (SRS: AVL01–AVL05, AVL08).

Các luồng được phép hoạt động ngoại tuyến phải giữ thời điểm thao tác, đánh dấu dữ liệu chưa đồng bộ và chống gửi trùng khi có mạng trở lại. Tác vụ nền có checkpoint hoặc lease để phục hồi sau lỗi. SRS đặt mục tiêu khôi phục dữ liệu giao dịch lõi với RPO không quá 1 giờ và RTO không quá 4 giờ; lịch sao lưu và thử phục hồi phải được vận hành, kiểm chứng riêng (SRS: AVL06–AVL07, BAK01–BA04).

2.6.5. Khả năng bảo trì

Khả năng bảo trì dựa trên việc tách giao diện khỏi API, giữ logic nghiệp vụ quan trọng phía máy chủ và dùng @bduck/shared-types cho enum, trạng thái và payload lỗi.

Thay đổi hợp đồng không tương thích cần version hoặc giai đoạn chuyển tiếp; thay đổi dữ liệu cần migration/backfill có dry-run, checkpoint và phương án hoàn nguyên (SRS: TC01–TC03, SCA06, BAK06).

Cấu hình, feature flag và bí mật phải được quản lý theo môi trường. Pipeline phát hành cần chạy lint, typecheck và các kiểm thử phân quyền, danh tính, Firestore Rules cùng tích hợp liên quan trước khi build hoặc triển khai; đây là công kiểm soát đặc biệt quan trọng vì SRS ghi nhận cấu hình build frontend hiện bỏ qua lỗi lint và TypeScript (SRS: TC08, TC10, TC16, SEC11).

2.7. Ràng buộc và giả định thiết kế

Ràng buộc nền tảng từ SRS gồm monorepo pnpm 9.15.4, Node.js từ phiên bản 22 và TypeScript; frontend Next.js/React ở chế độ standalone, backend Node.js/Express cung cấp REST API và gói @bduck/shared-types làm hợp đồng dùng chung. Thiết kế phải dựa trên API và quy ước của phiên bản framework thực tế trong mã nguồn, không giả định khả năng của phiên bản khác (SRS: TC01–TC03).

Firestore là kho dữ liệu nghiệp vụ chính, Firebase Authentication phục vụ định danh và Cloud Storage lưu tệp/chứng từ. Backend được đóng gói Docker và SRS định hướng triển khai trên Google Cloud Run tại asia-southeast1. Thiết kế phải tính đến truy vấn theo cơ sở, index, transaction và Firestore Rules (SRS: TC04–TC06).

Các luồng phụ thuộc MISA, JPOS/JoyWorld, Brevo, VietQR và Open API cần hợp đồng trao đổi, xác thực, timeout, idempotency và đối soát riêng. Tác vụ hóa đơn/voucher dài hạn cần hàng đợi hoặc worker được bảo vệ. Thời điểm nghiệp vụ và đồng bộ thống nhất theo múi giờ Asia/Ho_Chi_Minh (SRS: TC11–TC13).

Thiết kế giả định đơn vị vận hành cấp tài khoản dịch vụ, bí mật, IAM, mạng và môi trường Firebase/GCP; các phòng ban xác nhận quy tắc, phạm vi cơ sở và ánh xạ định danh. Vì chất lượng và tính sẵn sàng của dịch vụ ngoài không thuộc JPULSE, các luồng phụ thuộc cần trạng thái lỗi, retry và đối soát. Các giả định này phải được xác nhận trước triển khai.

Chương 3: Tổng quan và chiến lược kiến trúc

3.1. Nguyên tắc thiết kế

JPULSE ưu tiên phân tách trách nhiệm và bảo toàn tính đúng đắn của dữ liệu nghiệp vụ. Những nguyên tắc sau rút ra từ ranh giới hệ thống, ràng buộc kỹ thuật và yêu cầu chất lượng đã nêu trong SRS (Mục 2.4–2.6; Chương 7).

Phân tách trách nhiệm: Giao diện web trình bày trạng thái và tiếp nhận thao tác; API thực thi quy tắc nghiệp vụ, phân quyền, chuyển trạng thái và điều phối tích hợp; Firestore và Storage lưu trạng thái bền vững. Các thay đổi nghiệp vụ quan trọng phải đi qua API để có cùng một điểm kiểm soát quyền, kiểm tra đầu vào và ghi audit.

Quyền tối thiểu theo phạm vi: Mọi truy cập dữ liệu và lệnh xử lý phải gắn với định danh, quyền theo vai trò và phạm vi cơ sở. Khi thiếu hoặc sai ngữ cảnh quyền, hệ thống từ chối mặc định. Đường truy cập trực tiếp từ web đến Firestore/Storage chỉ được dùng trong phạm vi được Firebase Security Rules và chính sách lưu trữ cho phép.

Ranh giới dữ liệu và chống trùng: JPULSE sở hữu trạng thái quy trình và dữ liệu nội bộ; hệ thống ngoài giữ quyền đối với kết quả do họ phát hành. Lệnh có thể gửi lại phải có định danh ổn định hoặc khóa chống trùng. Với kết quả chưa xác định từ MISA, JPOS hay dịch vụ khác, JPULSE lưu trạng thái để đối soát trước khi tiếp tục xử lý.

Xử lý nền có thể phục hồi: Tác vụ dài hạn như phát hành hóa đơn hàng loạt hoặc tạo voucher được tiếp nhận và theo dõi bằng trạng thái bền vững, có retry và khả năng phục hồi. API phản hồi mã công việc thay vì giữ một kết nối chờ đến khi toàn bộ tác vụ hoàn tất.

Hợp đồng và cấu hình nhất quán: Các enum, trạng thái và payload lỗi được chia sẻ qua @bduck/shared-types. Cấu hình môi trường, bí mật và feature flag tách khỏi mã nguồn; thay đổi hợp đồng không tương thích cần có giai đoạn chuyển tiếp và được kiểm tra trong pipeline phát hành.

3.2. Kiểu kiến trúc được lựa chọn

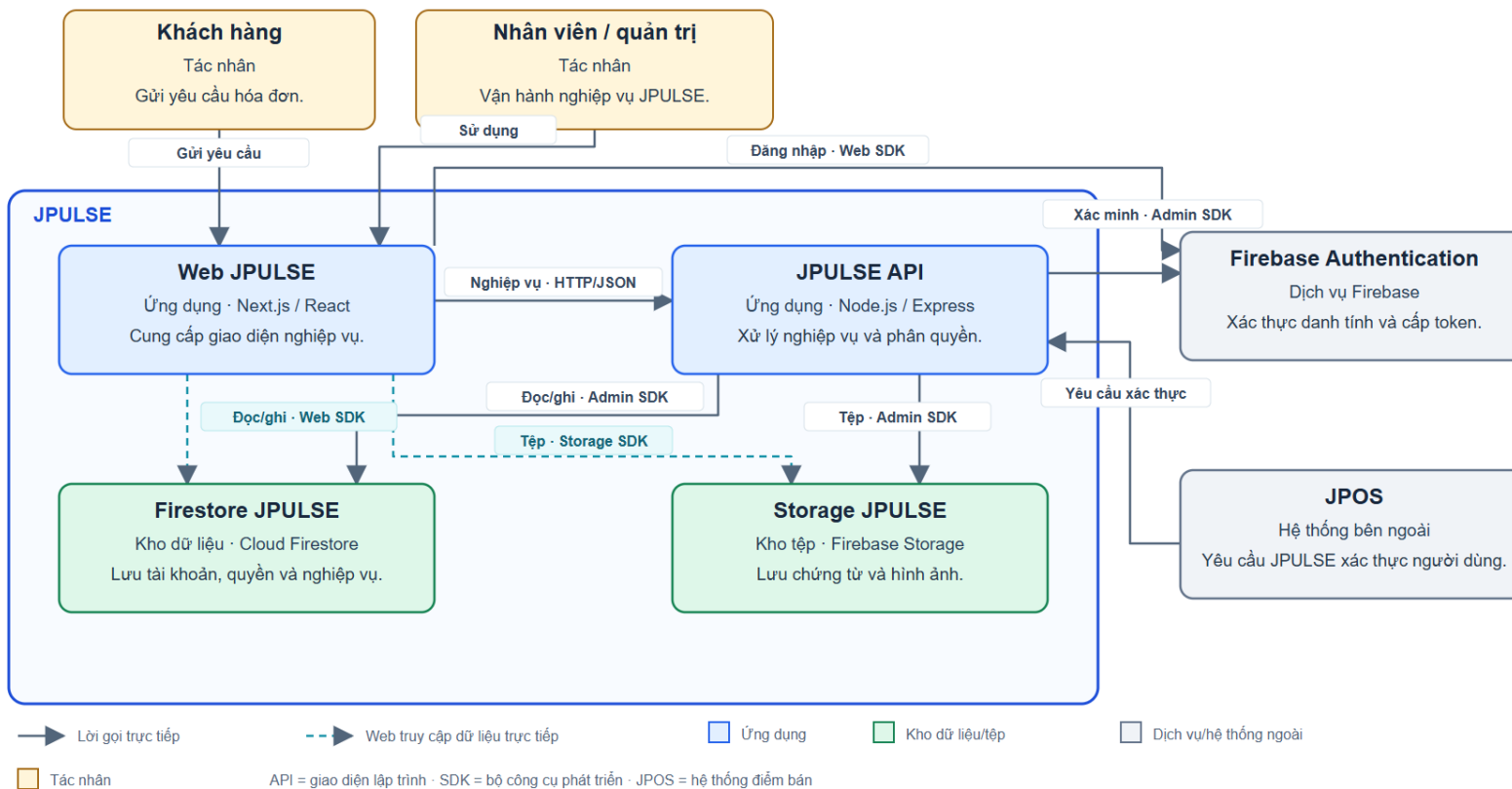
JPULSE được tổ chức thành ứng dụng web, dịch vụ API và các kho dữ liệu/tệp trên nền tảng Firebase. Web JPULSE dùng Next.js/React để cung cấp giao diện nghiệp vụ; JPULSE API dùng Node.js/Express để xử lý lệnh, phân quyền và tích hợp. Firestore lưu tài khoản, quyền và dữ liệu nghiệp vụ; Firebase Storage lưu chứng từ, hình ảnh và tệp. Firebase Authentication là dịch vụ xác thực danh tính bên ngoài ranh giới ứng dụng JPULSE.

Web trao đổi nghiệp vụ với API qua HTTP/JSON. Một số chức năng đọc dữ liệu thời gian thực và thao tác hợp được web thực hiện qua Firebase Web SDK, dưới sự kiểm soát của Security Rules; các lệnh tạo/sửa dữ liệu nghiệp vụ đi qua API. API sử dụng Firebase Admin SDK để xác minh danh tính, truy cập Firestore/Storage và thực thi quy tắc phía máy chủ. JPOS tương tác với JPULSE qua API; các tích hợp khác được mô tả ở những góc nhìn và luồng xử lý tương ứng.

Ở mức C4, web, API, Firestore và Storage là các container logic: ứng dụng hoặc kho dữ liệu có trách nhiệm riêng, không đồng nghĩa với Docker container. Cách đóng gói và triển khai trên hạ tầng được trình bày tại Chương 6; Hình 3-1 tập trung vào ranh giới và những kết nối trực tiếp quan trọng của hệ thống.

JPULSE · Container Diagram (C4 Level 2)

Ranh giới logic, ứng dụng, kho dữ liệu và kết nối trực tiếp



Hình 3-1: Sơ đồ container (C4 cấp 2) của JPULSE

3.4. Các thành phần chính và trách nhiệm

3.4.1. Giao diện web JPULSE

Web JPULSE là ứng dụng Next.js/React trong apps/fe-wms, cung cấp giao diện qua trình duyệt cho nhân viên và người quản trị. Các tuyến giao diện hiện có bao phủ dashboard, kho và sản phẩm, nhập xuất và điều chuyển, kiểm kê, nhân sự và chấm công, doanh thu và chi phí, hóa đơn điện tử, voucher, quản trị tài khoản/quyền cùng các chức năng cấu hình. Trang yêu cầu hóa đơn bằng token phục vụ khách hàng ở phạm vi công khai được tách khỏi khu vực quản trị. Giao diện quản trị hướng đến desktop; các luồng hiện trường cần dùng trên thiết bị di động được thiết kế theo yêu cầu Mobile Web của SRS. JPOS là hệ thống POS độc lập, không thuộc ứng dụng Web JPULSE (SRS, Mục 1.2 và 2.5).

Trách nhiệm giao diện: Thành phần web chịu trách nhiệm điều hướng, hiển thị dữ liệu và trạng thái quy trình, thu thập đầu vào, hỗ trợ lọc/tìm kiếm, phản hồi thao tác và trình bày lỗi cho người dùng. Web có thể kiểm tra định dạng và điều kiện đơn giản để giảm thao tác sai, nhưng quy tắc nghiệp vụ, quyền thực hiện, tính hợp lệ của trạng thái và thay đổi dữ liệu có tác động quan trọng phải được API kiểm tra lại. Các màn hình dùng chung kiểu dữ liệu và trạng thái cốt lõi từ @bduck/shared-types để giữ cách diễn giải nhất quán với máy chủ.

Xác thực và phân quyền: Người dùng đăng nhập bằng Firebase Web SDK; web gửi ID token tới JPULSE API để thiết lập phiên và nhận thông tin tài khoản, vai trò cùng phạm vi truy cập. Khu vực dashboard kiểm tra trạng thái đăng nhập và quyền đối với trang, chuyển người chưa đăng nhập đến trang đăng nhập, hiển thị trang 403 khi không đủ quyền và áp dụng màn hình khóa MFA khi cần. Việc ẩn hoặc khóa thao tác trên web chỉ là lớp hướng dẫn người dùng; API vẫn là điểm quyết định quyền và phạm vi cơ sở cho từng yêu cầu (SRS: SEC01–SEC03).

Giao tiếp và dữ liệu: Lệnh tạo/sửa dữ liệu nghiệp vụ được web gửi tới JPULSE API qua HTTP/JSON để máy chủ kiểm tra đầu vào, quyền, trạng thái và audit. Một số màn hình đọc dữ liệu Firestore theo thời gian thực bằng Firebase Web SDK với truy vấn giới hạn theo cơ sở; các thao tác ghi trực tiếp chỉ áp dụng cho phạm vi hẹp được Security Rules cho phép, không thay thế API đối với mutation nghiệp vụ. Web có luồng tải tệp qua Firebase Storage SDK và luồng sử dụng URL/chính sách tải do API cấp; quyền đối với bucket và tệp phải được kiểm soát phù hợp với từng luồng (SRS: TC05–TC06).

Trải nghiệm và trạng thái: Giao diện cần thể hiện trạng thái đang tải, đang xử lý, thành công, lỗi và chưa đồng bộ một cách rõ ràng. Với tác vụ nền, web hiển thị mã/trạng thái và tiến độ thay vì giữ kết nối chờ; lỗi nhập liệu đặt cạnh thao tác liên quan, còn lỗi hệ thống có thông điệp có thể xử lý mà không lộ chi tiết kỹ thuật. Bố cục phải phù hợp màn hình quản trị mật độ dữ liệu cao và các thao tác cảm ứng tại hiện trường; thuật ngữ, ngày giờ và trạng thái phải thống nhất với SRS, bao gồm các yêu cầu về khả năng tiếp cận và phản hồi khi mạng không ổn định (SRS: P02, USA01–USA08).

3.4.2. API nghiệp vụ JPULSE

JPULSE API là ứng dụng Node.js/Express viết bằng TypeScript trong apps/be-wms. API cung cấp các tuyến REST/JSON cho xác thực, tài khoản và vai trò, cơ sở và kho, sản phẩm, nhập xuất và điều chuyển, kiểm kê, nhân sự, doanh thu, chi phí, hóa đơn, POS và voucher. Trong mã nguồn, tuyến API tiếp nhận yêu cầu và áp dụng middleware; service thực thi chính sách nghiệp vụ; repository hoặc client tích hợp truy cập dữ liệu và hệ thống ngoài. Trang yêu cầu hóa đơn công khai sử dụng tuyến và cơ chế xác thực riêng với khu vực quản trị (SRS: TC03; Chương 4 và 6).

Kiểm soát nghiệp vụ và truy cập: API xác minh định danh hoặc phiên, xây dựng ngữ cảnh truy cập, kiểm tra quyền theo vai trò và Office Scope, kiểm tra schema và trạng thái nghiệp vụ trước khi ghi dữ liệu. Những thay đổi quan trọng như biến động tồn kho, phê duyệt, sổ dư nghỉ phép và phát hành hóa đơn phải bảo toàn tính nhất quán, lưu lịch sử xử lý và chống gửi trùng khi yêu cầu có thể thử lại. Middleware hiện có áp dụng Helmet, CORS theo danh sách cho phép, giới hạn tốc độ và giới hạn kích thước JSON; quyền ở giao diện không thay thế kiểm tra phía API (SRS: BC01–BC07, SEC01–SEC10).

Điều phối và vận hành: API dùng Firebase Admin SDK để xác minh token và truy cập Firestore/Storage bằng quyền máy chủ, đồng thời gọi các dịch vụ bên ngoài qua client hoặc adapter riêng. Lệnh tích hợp kéo dài được giao cho tác vụ nền hoặc hàng đợi khi cấu hình cho phép; trạng thái job, khóa chống trùng và kết quả từng phần được lưu bền vững để có thể retry hoặc đối soát. Dịch vụ phải không phụ thuộc trạng thái chỉ nằm trong bộ nhớ một instance, phù hợp với cách mở rộng ngang của Cloud Run (SRS: TC04–TC05, TC11–TC13, SCA01–SCA05).

3.4.3. Gói kiểu dữ liệu và hợp đồng dùng chung

Gói @bduck/shared-types trong packages/shared-types là hợp đồng TypeScript được Web JPULSE và JPULSE API dùng chung. Gói tập hợp enum, trạng thái, kiểu dữ liệu và payload cốt lõi của các miền như người dùng và quyền, kho, phiếu, nhân sự, chấm công, nghỉ phép, hóa đơn, POS và voucher; một số hợp đồng có schema kiểm tra dữ liệu. Nhờ dùng cùng định nghĩa, hai ứng dụng giảm sai khác khi diễn giải mã trạng thái, trường dữ liệu và kết quả nghiệp vụ (SRS: TC01, SCA06).

Ranh giới và thay đổi: Gói này được build và nhập như dependency trong npm workspace; nó không chạy thành dịch vụ độc lập và không lưu dữ liệu. Thay đổi hợp đồng cần được kiểm tra cùng frontend và backend. Khi đổi tên trường, enum hoặc ý nghĩa trạng thái theo cách không tương thích, thiết kế phải có migration hoặc giai đoạn chuyển tiếp để các phiên bản triển khai khác nhau không đọc sai dữ liệu hay phản hồi API (SRS: SCA06, COM06).

3.4.4. Kho dữ liệu và lưu trữ tệp

Firestore JPULSE là kho dữ liệu NoSQL chính cho tài khoản nghiệp vụ, vai trò, quyền và phạm vi cơ sở, danh mục, chứng từ, trạng thái quy trình, dữ liệu giao dịch, cấu hình và audit. Dữ liệu được tổ chức theo miền nghiệp vụ như SRS Chương 5; các truy vấn phải giới hạn theo quyền và cơ sở, dùng index và phân trang khi tập dữ liệu lớn. Các thay đổi liên quan đồng thời đến nhiều bản ghi, đặc biệt là tồn kho hoặc số dư, cần transaction/batch và kiểm soát retry để không tạo trạng thái dở dang (SRS: TC05–TC07, AVL03–AVL04).

Đường truy cập dữ liệu: Web có thể đọc một số dữ liệu thời gian thực bằng Firebase Web SDK và thực hiện các ghi hợp được Firestore Security Rules cho phép. Rules từ chối mặc định và kiểm tra bản ghi quyền hiện hành; các mutation nghiệp vụ quan trọng đi qua JPULSE API dùng Admin SDK để áp dụng chính sách, kiểm tra trạng thái và ghi audit. Firebase Authentication cung cấp định danh, còn trạng thái tài khoản, vai trò và quyền nghiệp vụ thuộc dữ liệu JPULSE trong Firestore.

Lưu trữ tệp: Storage JPULSE dùng Firebase Cloud Storage để lưu chứng từ, hợp đồng, hình ảnh bằng chứng và tệp nghiệp vụ. Firestore lưu metadata, định danh, trạng thái và tham chiếu tới tệp; nội dung tệp nằm trong Storage. Tùy luồng, web tải tệp bằng Storage SDK hoặc dùng URL/chính sách tải do API cấp, còn API quản lý tệp bằng Admin SDK. Thiết kế phải kiểm tra quyền sở hữu, loại và kích thước tệp, đường dẫn lưu, thời hạn URL và chính sách bucket theo môi trường; bản sao lưu phải bao gồm cả dữ liệu và tệp (SRS: TC05–TC06, SEC09, BAK01).

3.4.5. Dịch vụ tích hợp bên ngoài

Firestore Authentication là dịch vụ xác thực danh tính và cấp token cho người dùng; JPULSE API xác minh token/phần trước khi áp dụng quyền nghiệp vụ. Firebase Cloud Messaging chuyển thông báo đẩy đến thiết bị. Google Cloud Tasks có thể phân phối tác vụ hóa đơn hoặc voucher và gọi lại worker được bảo vệ; Cloud Scheduler được SRS nêu cho các công việc định kỳ theo cấu hình triển khai. Các dịch vụ này hỗ trợ JPULSE nhưng không nắm quy tắc nghiệp vụ hoặc quyền truy cập của người dùng (SRS: INT-01, INT-10).

JPOS là hệ thống điểm bán độc lập. JPULSE trao đổi dữ liệu sản phẩm, thiết bị, định danh hoặc yêu cầu xác thực qua API theo từng luồng đã xác nhận. CityfunS/JoyWorld cung cấp nguồn dữ liệu doanh thu, đơn hàng và tồn đối tác; các thao tác ghi hoặc hoàn tiền chỉ được bật theo cấu hình và quyền. SRS cũng nhắc hệ thống ERP cũ, nhưng hợp đồng tích hợp và trạng thái vận hành cụ thể cần được xác minh trước khi coi đó là luồng đang chạy (SRS: INT-04–INT-07).

MISA meInvoice là nguồn kết quả phát hành và trạng thái hóa đơn điện tử; JPULSE chuẩn hóa dữ liệu, gửi yêu cầu, lưu định danh giao dịch và đối soát kết quả. Brevo chuyển phát email giao dịch, OTP và voucher; VietQR cung cấp dữ liệu tra cứu doanh nghiệp để hỗ trợ nhập thông tin hóa đơn. Mapbox được web sử dụng cho bản đồ hoặc tuyến điều chuyển khi có cấu hình phù hợp. Mỗi kết nối chỉ trao đổi dữ liệu cần thiết và tuân theo hợp đồng API hoặc SMTP của dịch vụ tương ứng (SRS: INT-02–INT-03, INT-08–INT-09).

Kiểm soát ranh giới tích hợp: Thông tin xác thực và secret của các tích hợp máy chủ phải nằm trong cấu hình được bảo vệ, không chuyển ra bundle trình duyệt. API đặt timeout, phân loại lỗi có thể retry, lỗi kết thúc và kết quả chưa xác định; các thao tác có tác động tài chính hoặc tồn kho dùng khóa chống trùng, lưu trạng thái và hỗ trợ đối soát thủ công. Khi dịch vụ ngoài không sẵn sàng, JPULSE phải thông báo đúng trạng thái và để các nghiệp vụ độc lập tiếp tục hoạt động (SRS: TC08, TC13, AVL04–AVL08).

Bảng 3-1: Các thành phần chính của hệ thống

Thành phần	Trách nhiệm	Giao tiếp	Ghi chú
Web JPULSE	Giao diện và trạng thái thao tác	HTTP/JSON; Firebase Web SDK	Next.js/ React
JPULSE API	Quy tắc, quyền, dữ liệu và tích hợp	REST; Firebase Admin SDK	Node.js/ Express
Gói shared-types	Kiểu, enum và hợp đồng dùng chung	Được nhập bởi web và API	Gói mã nguồn
Firestore JPULSE	Dữ liệu nghiệp vụ, quyền và audit	Web SDK có phạm vi; Admin SDK	Kho dữ liệu
Storage JPULSE	Chứng từ, hình ảnh và tệp	Storage SDK; API/URL ký	Kho tệp
Firebase Authentication	Xác thực danh tính và token	Web SDK; Admin SDK	Dịch vụ ngoài
JPOS/CityfunS	Nguồn hoặc đích dữ liệu bán hàng	API theo hợp đồng tích hợp	Hệ thống ngoài
Dịch vụ khác	Hóa đơn, email, tra cứu, tác vụ nền	API/SMTP; Cloud Tasks	MISA, Brevo...

3.5. Cách các thành phần giao tiếp

Web và API: Web JPULSE gửi các lệnh nghiệp vụ tới JPULSE API bằng HTTP(S) với dữ liệu JSON; phản hồi trả về kết quả, trạng thái hoặc lỗi để giao diện cập nhật thao tác. Các hợp đồng dữ liệu cốt lõi được dùng chung qua gói @bduck/shared-types, còn API là nơi kiểm tra dữ liệu đầu vào, quyền và quy tắc nghiệp vụ. Với tuyến cần đăng nhập, web chuyển thông tin xác thực do Firebase Authentication cấp; API xác minh định danh, nạp trạng thái tài khoản, vai trò, quyền và phạm vi cơ sở trước khi xử lý. Các tuyến công khai, như yêu cầu hóa đơn của khách hàng, dùng cơ chế token và giới hạn truy cập riêng theo luồng (SRS: TC01–TC03, SEC01–SEC06).

Xác thực và dữ liệu thời gian thực: Web dùng Firebase Web SDK để đăng nhập. Một số màn hình đọc dữ liệu Firestore theo thời gian thực và phạm vi được phép; các ghi trực tiếp từ trình duyệt chỉ dành cho các trường hợp hẹp mà Firestore Security Rules cho phép, như trạng thái thông báo hoặc tín hiệu truyền tệp LAN. Những lệnh làm thay đổi dữ liệu nghiệp vụ đi qua API để áp dụng cùng một chính sách quyền, kiểm tra trạng thái, giao dịch và audit. API truy cập Firestore bằng Firebase Admin SDK; quyền của Admin SDK không được chuyển xuống trình duyệt (SRS: TC05–TC07, SEC07–SEC10).

Giao tiếp với kho tệp: Tệp chứng từ và hình ảnh được trao đổi qua Storage JPULSE. Tùy nghiệp vụ, web tải tệp bằng Storage SDK theo chính sách truy cập hoặc bằng URL/chính sách tải có thời hạn do API cấp. API dùng Admin SDK để quản lý tệp, liên kết metadata trong Firestore với đối tượng trong Storage và kiểm tra quyền trước khi cấp quyền truy cập. Khi thao tác dữ liệu và tệp không thể nằm trong cùng một giao dịch, ứng dụng phải lưu trạng thái xử lý để phát hiện, thử lại hoặc dọn dẹp tệp thiếu tham chiếu (SRS: TC05–TC06, SEC09, AVL04).

Giao tiếp với hệ thống ngoài: JPULSE API gọi các hệ thống ngoài qua client hoặc adapter theo hợp đồng của từng bên: JPOS cho các luồng điểm bán, CityfunS/JoyWorld cho dữ liệu bán hàng, MISA meInvoice cho hóa đơn, VietQR cho tra cứu doanh nghiệp và Brevo cho email. Web gọi Mapbox trực tiếp để hiển thị bản đồ khi tính năng được cấu hình. Mỗi kết nối chỉ truyền dữ liệu cần thiết, dùng thông tin xác thực phù hợp, đặt timeout và chuyển lỗi dịch vụ ngoài thành trạng thái nghiệp vụ có thể hiểu được. Quan hệ trao đổi với ERP cũ được SRS đề cập nhưng chiều gọi và hợp đồng triển khai cần xác minh (SRS: INT-02–INT-09).

Tác vụ nền và thông báo: API có thể tạo Cloud Task cho hóa đơn hoặc voucher khi hàng đợi được cấu hình. Worker xác thực lời gọi, chống xử lý trùng và ghi kết quả hoặc lỗi để đối soát. API gửi thông báo đẩy qua Firebase Cloud Messaging; web đọc trạng thái tác vụ từ dữ liệu JPULSE (SRS: INT-01, INT-10, TC11–TC13).

3.6. Lý do chọn giải pháp kiến trúc

Tách giao diện và xử lý nghiệp vụ: Cách tổ chức Web JPULSE, JPULSE API và các kho dữ liệu phản ánh cả phạm vi nghiệp vụ lẫn nền tảng đang sử dụng. Web tập trung vào hiển thị và thao tác; API tập trung quy tắc, quyền và tích hợp; Firestore cùng Storage lưu trạng thái bền vững. Ranh giới này cho phép thay đổi giao diện mà không nhân bản quy tắc nghiệp vụ, đồng thời giúp nhóm phát triển xác định nơi xử lý khi bổ sung các miền kho, nhân sự, doanh thu hoặc hóa đơn. Nó cũng phù hợp ràng buộc monorepo TypeScript, Next.js/React, Node.js/Express và Firebase đã nêu ở Mục 2.7 (SRS: TC01–TC06).

Ưu tiên tính đúng đắn và quyền truy cập: Đặt các lệnh tạo, sửa và phê duyệt quan trọng tại API tạo một điểm kiểm soát thống nhất cho xác thực, quyền theo vai trò và phạm vi cơ sở, hợp lệ hóa dữ liệu, chuyển trạng thái và audit. Điều này cần thiết vì dữ liệu kho, sổ dư, hóa đơn và hồ sơ nhân sự có thể bị truy cập từ nhiều màn hình hoặc tác nhân; kiểm tra tại web không đủ để bảo vệ chúng. Đường đọc Firestore thời gian thực từ web chỉ phục vụ những nhu cầu phản hồi nhanh đã được Security Rules giới hạn. Sự kết hợp này đáp ứng yêu cầu bảo mật mà vẫn giữ trải nghiệm cập nhật kịp thời; đổi lại, quyền ở API và Rules phải được kiểm thử cùng nhau để tránh lệch chính sách (SRS: SEC01–SEC10, BC01–BC07).

Lưu trữ theo trách nhiệm dữ liệu: Firestore phù hợp với mô hình dữ liệu theo miền và các màn hình cần đọc cập nhật gần thời gian thực; Storage tách nội dung tệp khỏi metadata nghiệp vụ. Việc dùng dịch vụ được quản lý giảm phần hạ tầng phải tự vận hành và phù hợp định hướng triển khai trên Google Cloud. Tuy nhiên, Firestore đòi hỏi truy vấn có phạm vi, index và phân trang; thay đổi liên quan nhiều bản ghi phải dùng transaction/batch cùng cơ chế chống gửi trùng. Tệp và metadata không có một giao dịch chung nên các luồng tải tệp cần trạng thái trung gian và xử lý bù khi thất bại (SRS: TC05–TC07, SCA03, AVL03–AVL04).

Mở rộng và phục hồi tác vụ: API không giữ trạng thái nghiệp vụ chỉ trong bộ nhớ một instance để có thể chạy trên Cloud Run và mở rộng theo tải. Các công việc dài như phát hành hóa đơn hàng loạt hoặc xử lý voucher được theo dõi bằng trạng thái bền vững và có thể chuyển sang Cloud Tasks khi được cấu hình. API vì vậy không phải giữ yêu cầu người dùng chờ hệ thống ngoài; worker có thể thử lại sau lỗi và đối soát kết quả chưa xác định. Lựa chọn này phục vụ mục tiêu thời gian phản hồi, tính sẵn sàng và khả năng phục hồi, nhưng yêu cầu khóa chống trùng, giới hạn số lần thử, giám sát hàng đợi và thao tác xử lý thủ công cho trường hợp không thể tự khôi phục (SRS: P01–P04, SCA01–SCA05, AVL01–AVL08).

Giảm phụ thuộc khi thay đổi: Gói @bduck/shared-types giữ enum, trạng thái và payload cốt lõi nhất quán giữa web và API; các client/adaptor tích hợp cô lập hợp đồng riêng của JPOS, CityfunS/JoyWorld, MISA và những dịch vụ khác khỏi quy tắc nghiệp vụ nội bộ. Nhờ vậy, thay đổi ở một giao diện bên ngoài có phạm vi tác động rõ hơn, còn các luồng khác có thể tiếp tục khi một dịch vụ tích hợp gặp lỗi. Đánh đổi là cần quản lý phiên bản hợp đồng, cấu hình theo môi trường và kiểm thử tương thích khi triển khai web, API hoặc adaptor vào những thời điểm khác nhau. Các quyết định và rủi ro cụ thể được đánh giá thêm ở Chương 7 (SRS: SCA06, TC08–TC13, COM06).

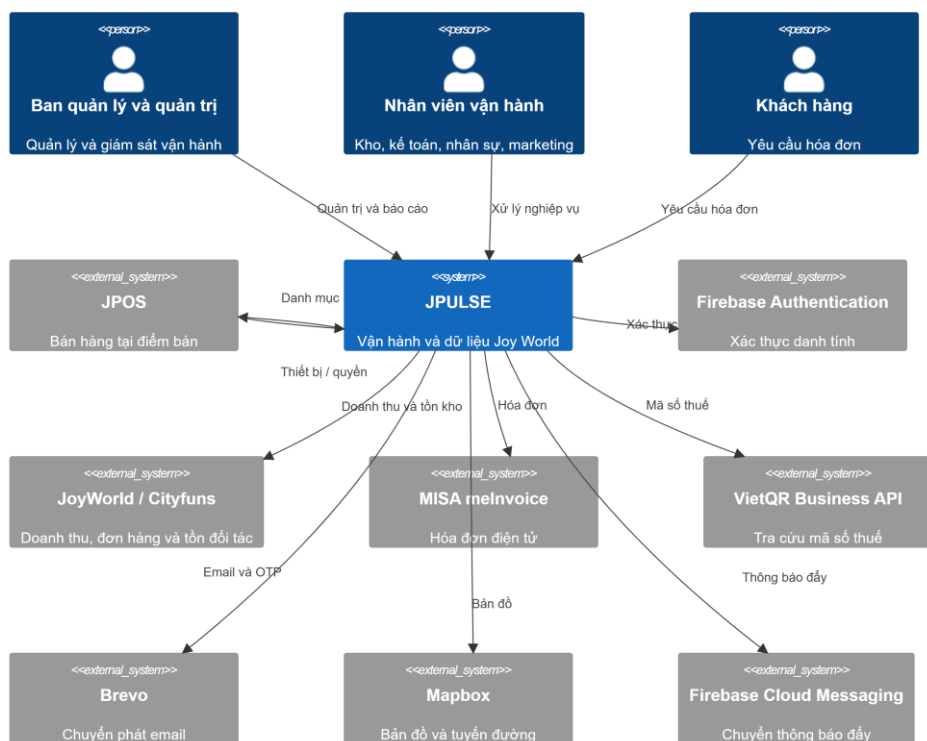
Chương 4: Các góc nhìn kiến trúc

4.1. Góc nhìn ngữ cảnh – C4 System Context

Góc nhìn ngữ cảnh xem JPULSE là một hệ thống phần mềm duy nhất trong hoạt động Joy World. Ban quản lý và quản trị dùng hệ thống để giám sát, phê duyệt và cấu hình; nhân viên vận hành xử lý nghiệp vụ theo quyền và cơ sở; khách hàng chỉ gửi yêu cầu hóa đơn trong luồng công khai. Hình 4-1 cho thấy những tác nhân này cùng các hệ thống do bên khác vận hành, không phân rã thành phần nội bộ của JPULSE (SRS, Mục 2.3–2.4).

Hình 4-1: Sơ đồ ngữ cảnh hệ thống JPULSE

Sơ đồ ngữ cảnh hệ thống JPULSE

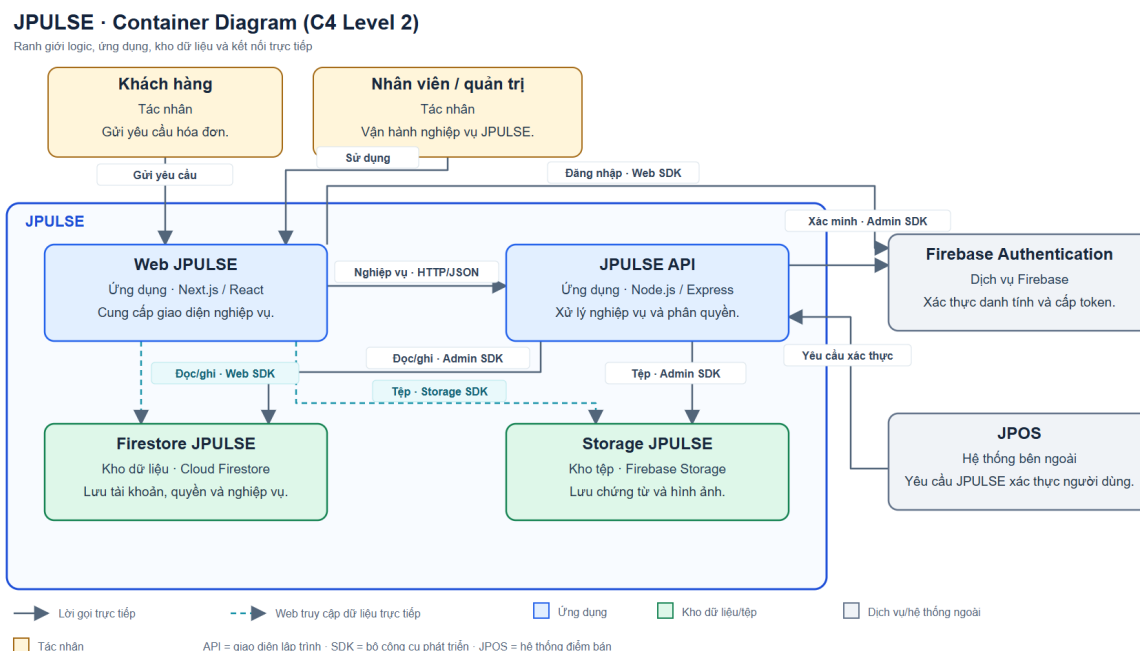


JPULSE trao đổi danh mục và thông tin liên quan điểm bán với JPOS; lấy dữ liệu bán hàng từ JoyWorld/CityfunS; dùng Firebase Authentication cho định danh, MISA meInvoice cho hóa đơn, VietQR cho tra cứu doanh nghiệp, Brevo cho email, Mapbox cho bản đồ và Firebase Cloud Messaging cho thông báo đẩy. Mũi tên trên hình chỉ bên khởi tạo tương tác; việc có một đường tích hợp trong mã không khẳng định nó đã được bật ở mọi môi trường. ERP cũ không có đường trao đổi kỹ thuật đã được xác minh nên không xuất hiện trên hình. Các trách nhiệm và ranh giới chi tiết được nêu ở Mục 2.3–2.4.

4.2. Góc nhìn ứng dụng và kho dữ liệu – C4 Container

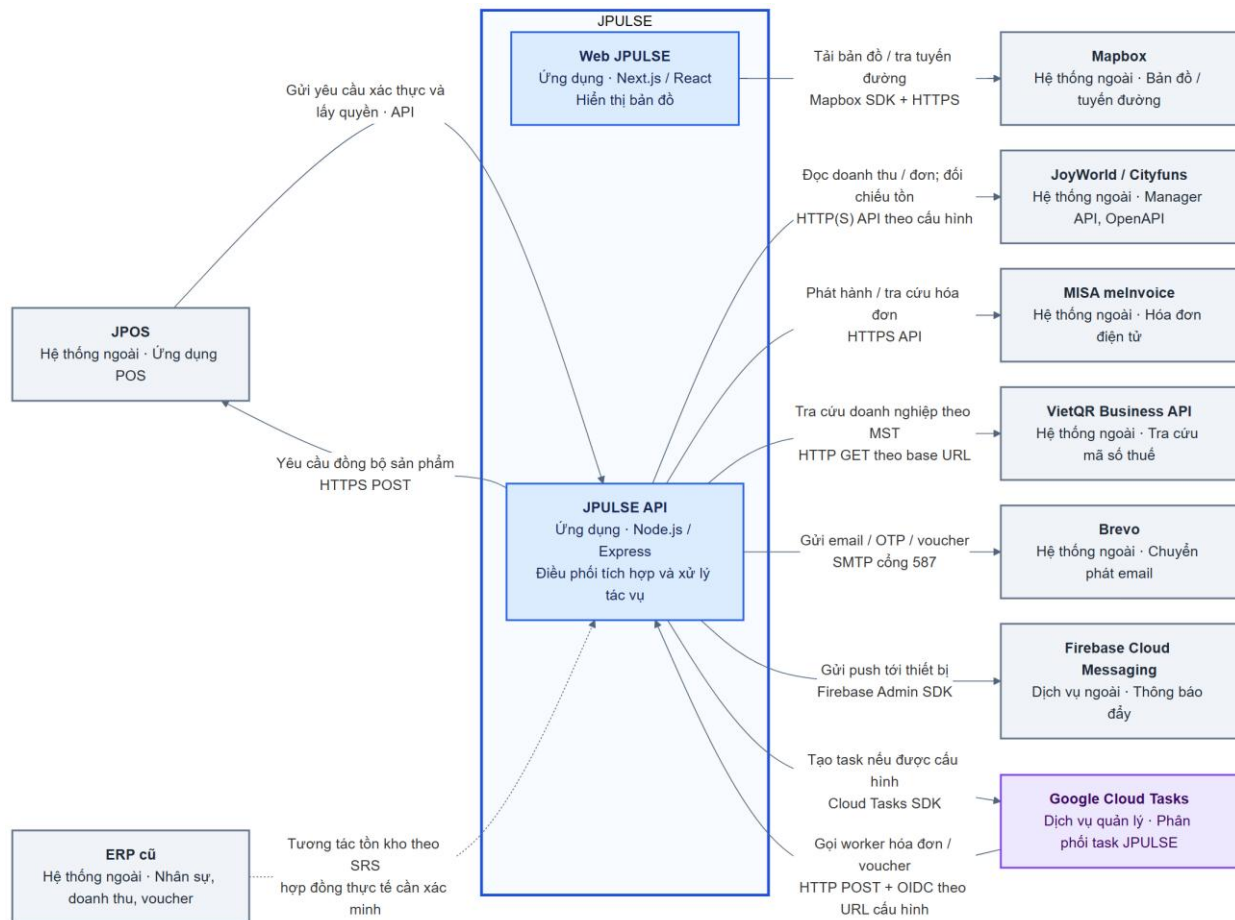
Hình 4-2 phân rã JPULSE thành hai ứng dụng và hai kho dữ liệu logic. Web JPULSE cung cấp giao diện; JPULSE API thực thi lệnh nghiệp vụ, quyền và tích hợp; Firestore lưu dữ liệu có cấu trúc; Storage lưu tệp. Firebase Authentication là dịch vụ xác thực bên ngoài ứng dụng. Trong mô hình C4, các khối này là container theo trách nhiệm chạy và lưu trữ, không đồng nghĩa với Docker container.

Hình 4-2: Sơ đồ container của JPULSE



Đường chính cho thao tác tạo và sửa là Web → API → Firestore/Storage, nơi API kiểm tra quyền và trạng thái trước khi ghi. Web cũng dùng Firebase Web SDK để đăng nhập, đọc một số dữ liệu thời gian thực hoặc tải tệp trong phạm vi Security Rules cho phép. API dùng Admin SDK phía máy chủ. Gói @bduck/shared-types là mã hợp đồng được hai ứng dụng nhập khi build, nên không là container chạy độc lập. Hình 4-2 cũng đã được dùng ở Mục 3.3 để trình bày kiến trúc tổng thể.

Hình 4-3: Sơ đồ tích hợp với hệ thống và dịch vụ bên ngoài



Hình 4-3 bổ sung các đường tích hợp không thể hiện hết trong sơ đồ container chính: API gọi hoặc nhận lời gọi từ JPOS, CityfunS/JoyWorld, MISA, VietQR, Brevo, FCM và Cloud Tasks; Web gọi Mapbox. Đường nét đứt với ERP cũ chỉ phản ánh quan hệ SRS đề cập, còn hợp đồng và trạng thái triển khai chưa được xác minh. API phải giữ bí mật tích hợp ở phía máy chủ, kiểm soát timeout và lưu trạng thái đối soát cho các lời gọi có thể có kết quả chưa xác định (SRS: INT-01–INT-10).

4.3. Góc nhìn thành phần – C4 Component

Góc nhìn thành phần chỉ phân rõ hai ứng dụng có cấu trúc bên trong cần giải thích. Các mô tả dưới đây nêu trách nhiệm và chiều phụ thuộc từ mã nguồn; kho biểu đồ hiện chưa có C4 Component Diagram riêng cho Web hoặc API. Vì vậy, không suy diễn các module thành những tiến trình triển khai độc lập.

4.3.1. Thành phần giao diện web

Trong Web JPULSE, lớp route và layout của Next.js định tuyến các khu vực dashboard, nghiệp vụ và trang yêu cầu hóa đơn công khai. Các feature component nhận dữ liệu, hiển thị trạng thái và thu thập thao tác; hook/store giữ trạng thái phiên, quyền, bộ lọc và dữ liệu hiển thị. Lớp API client gửi lệnh đến JPULSE API, còn thư viện Firebase phục vụ xác thực, listener Firestore và một số luồng tệp được cho phép. Các lớp này dùng hợp đồng @bduck/shared-types để diễn giải payload và trạng thái nhất quán (SRS: TC01–TC02, SEC01–SEC03).

Luồng phụ thuộc thông thường là route/component → hook hoặc client → JPULSE API hay Firebase SDK. Kiểm tra tại giao diện giúp người dùng biết thao tác hợp lệ và phản hồi lỗi sớm; nó không thay thế kiểm tra quyền, phạm vi cơ sở và trạng thái nghiệp vụ tại API. Ranh giới giữa truy cập Firebase trực tiếp và lệnh phải qua API cần được giữ rõ khi thêm màn hình mới.

4.3.2. Thành phần API nghiệp vụ

Trong JPULSE API, route tiếp nhận HTTP request và gắn middleware xác thực, giới hạn truy cập; controller chuẩn hóa đầu vào và phản hồi; service giữ quy tắc, trạng thái và điều phối giao dịch; repository hoặc Firebase Admin SDK đọc ghi Firestore/Storage. Client tích hợp bao bọc hợp đồng JPOS, MISA và các hệ thống ngoài; worker xử lý tác vụ được giao lại từ hàng đợi. Cách phân lớp này giữ quy tắc nghiệp vụ ở máy chủ, đồng thời cô lập thay đổi giao thức bên ngoài (SRS: TC03–TC05, TC11–TC13).

Chiều gọi chính là route/middleware → controller → service → repository hoặc client tích hợp. Middleware và service đều cần ngữ cảnh định danh, vai trò và Office Scope; service phải kiểm tra lại điều kiện nghiệp vụ ngay trước thay đổi dữ liệu. Các thao tác nhiều bước lưu trạng thái và audit để nhận biết phần đã thành công khi bước sau lỗi. C4 Component Diagram cho API vẫn cần được vẽ để thể hiện chính thức các quan hệ này; phần mô tả hiện tại là cơ sở cho hình đó.

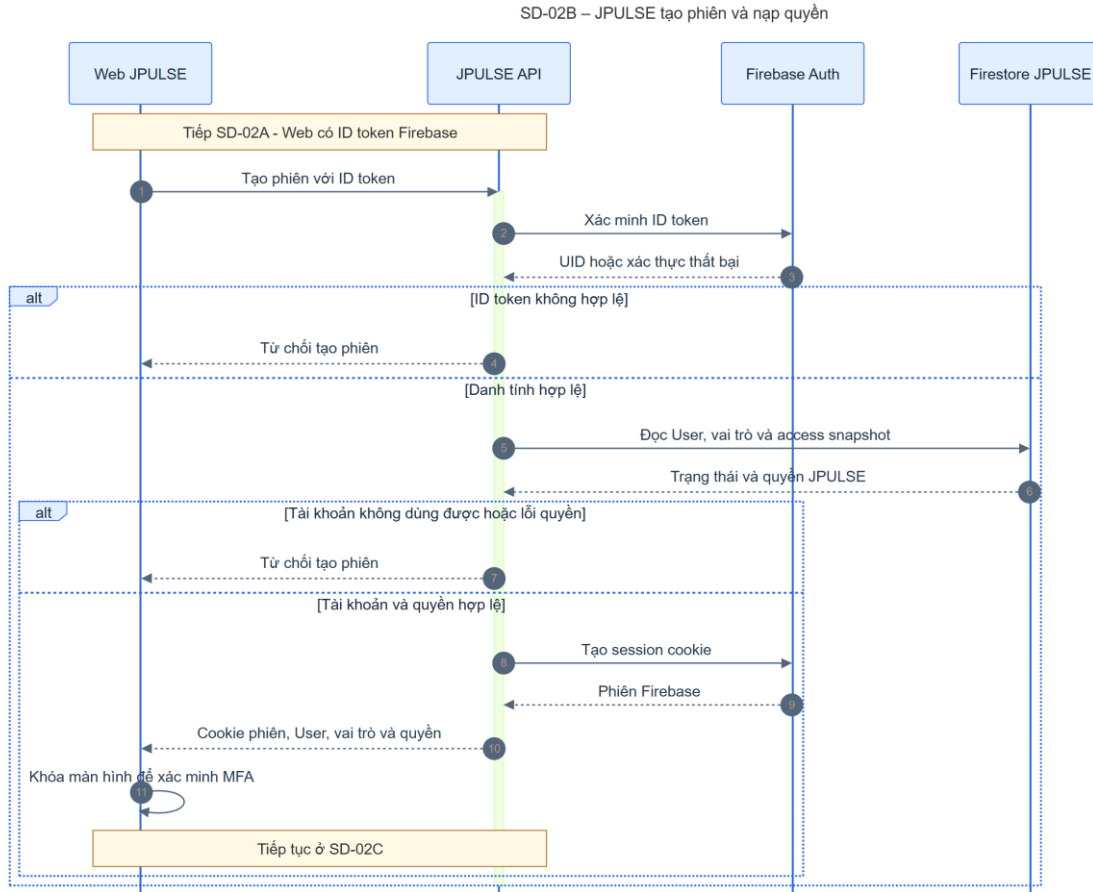
4.4. Góc nhìn động

Góc nhìn động chọn những sequence diagram thể hiện ranh giới xác thực, ghi dữ liệu kho và tương tác với hệ thống ngoài. Mỗi hình tập trung một bước có giá trị kiến trúc; những bước kẻ trước/sau được chỉ rõ bằng mã SD trong phần mô tả. Bộ hình đầy đủ nằm trong docs/architecture/sequence/. Mũi tên liền biểu diễn lời gọi, mũi tên đứt biểu diễn phản hồi hoặc dữ liệu đẩy.

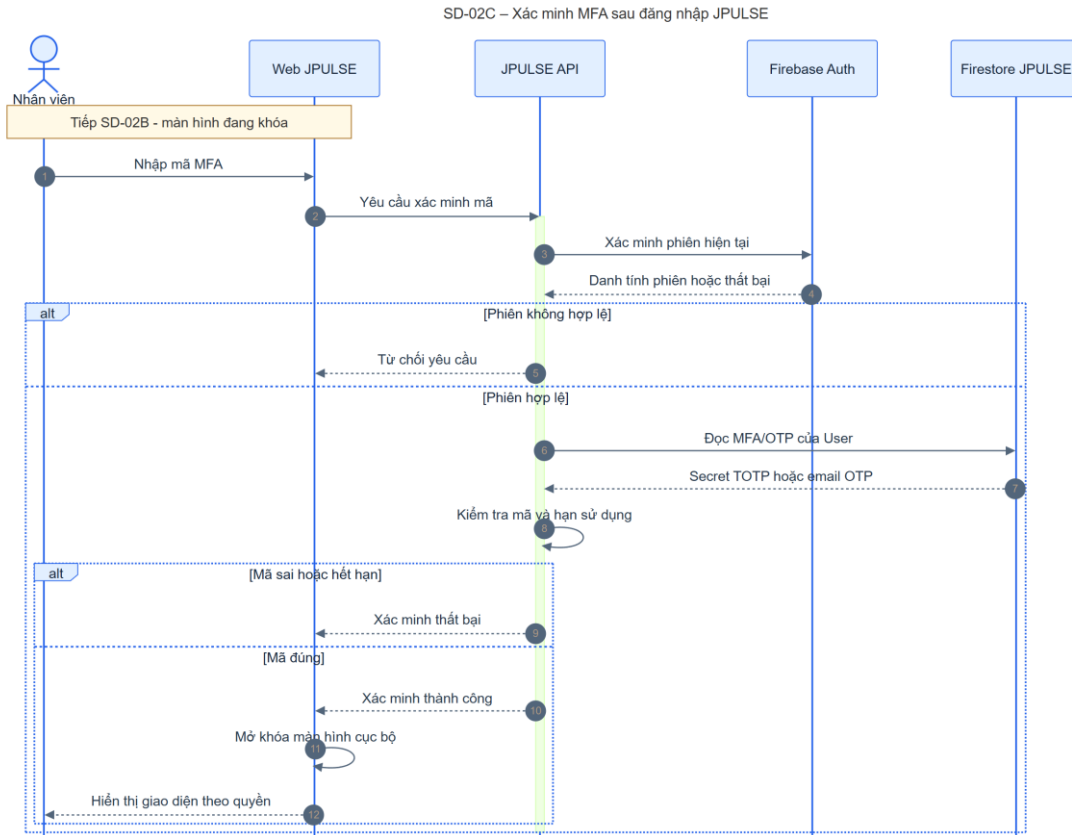
4.4.1. Luồng xác thực và phân quyền

Luồng đăng nhập Web bắt đầu bằng phân giải định danh và xác thực với Firebase Authentication ở SD-02A. Hình 4-4 tiếp tục từ ID token: API xác minh token, đọc tài khoản và quyền JPULSE trong Firestore, từ chối tài khoản không hợp lệ rồi mới tạo phiên. Firebase xác nhận danh tính; quyền nghiệp vụ và phạm vi cơ sở do JPULSE quyết định.

Hình 4-4: SD-02B tạo phiên và nạp quyền JPULSE



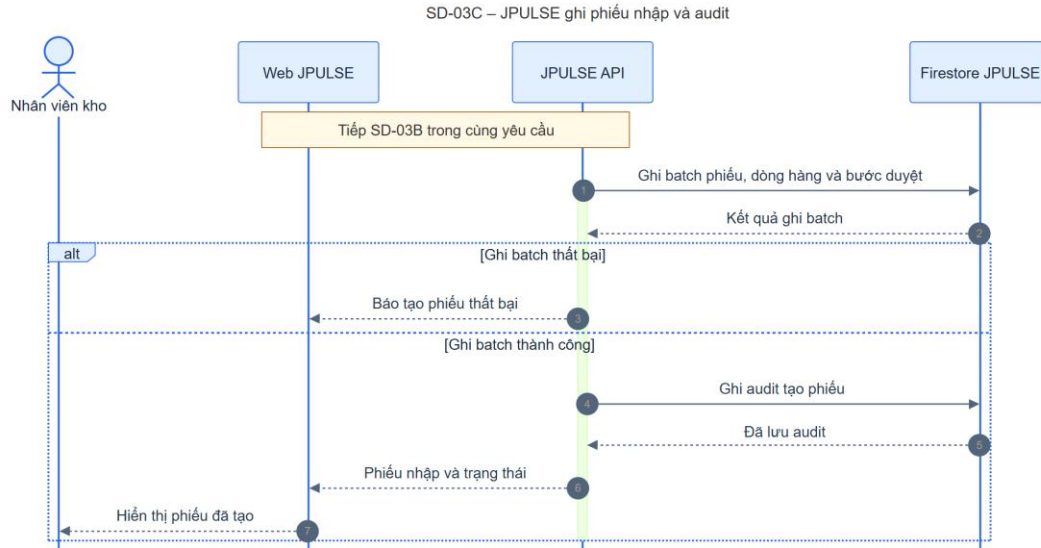
Nếu tài khoản yêu cầu MFA, Web giữ màn hình khóa sau khi tạo phiên. Hình 4-5 cho thấy API xác minh mã TOTP hoặc email OTP; mã sai/hết hạn không mở thao tác, còn mã hợp lệ cho phép Web tiếp tục. Trạng thái mở khóa ở sơ đồ là trạng thái giao diện sau khi API xác minh, không làm thay đổi nguyên tắc kiểm tra quyền tại API cho mỗi lệnh (SRS: SEC01–SEC03).

Hình 4-5: SD-02C xác minh MFA sau đăng nhập


4.4.2. Luồng nhập xuất và điều chuyển kho

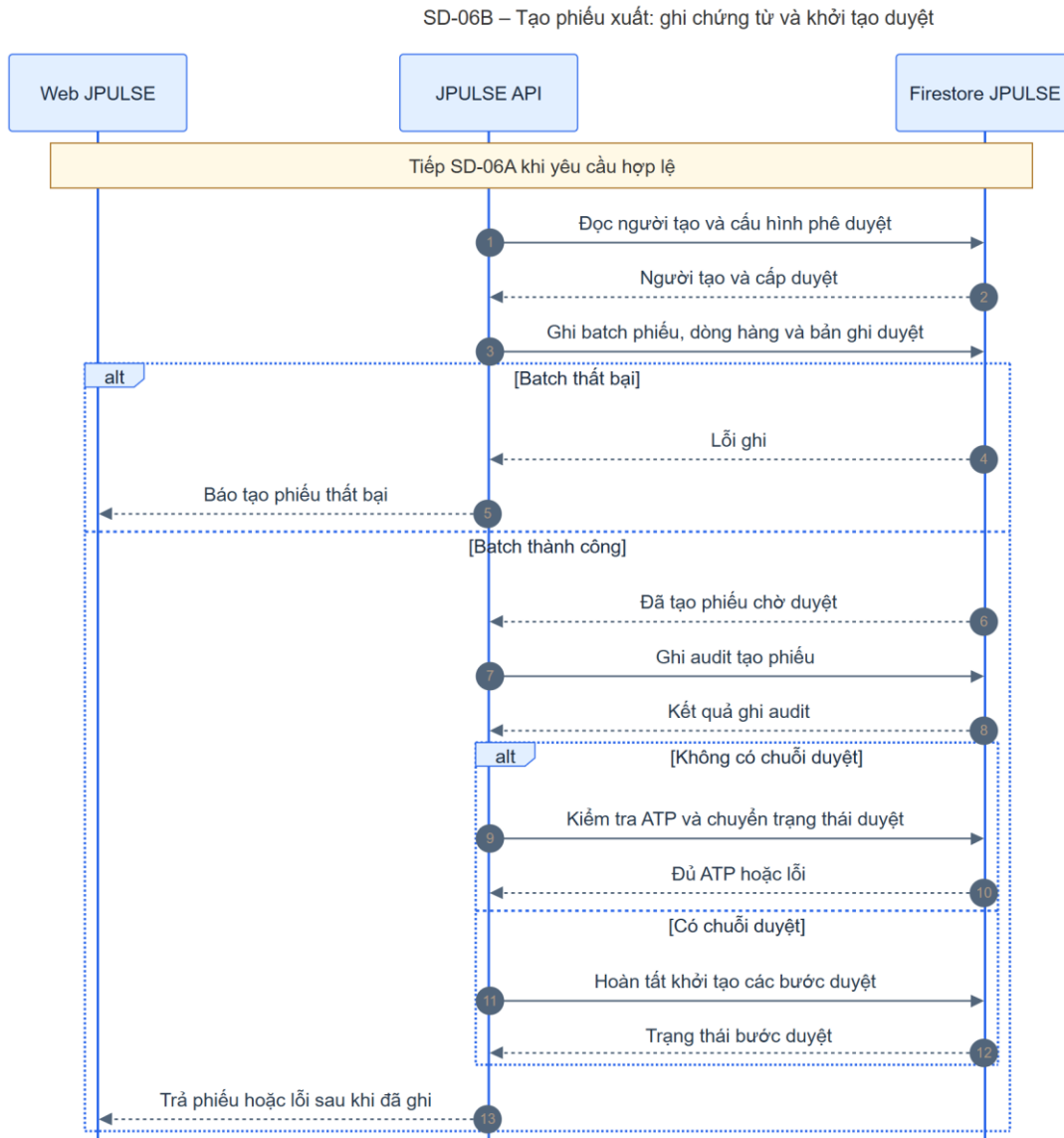
Với phiếu nhập, các bước SD-03A/B tải chứng từ và kiểm tra quyền, dữ liệu trước khi ghi. Hình 4-6 thể hiện API ghi phiếu, dòng hàng và bước duyệt bằng batch Firestore, sau đó ghi audit và trả trạng thái. Batch thất bại không tạo phiếu; audit được ghi sau batch nên lỗi ở bước audit cần được giám sát và đối soát riêng, không được mô tả là toàn bộ thao tác đã tự rollback.

Hình 4-6: SD-03C ghi phiếu nhập và audit



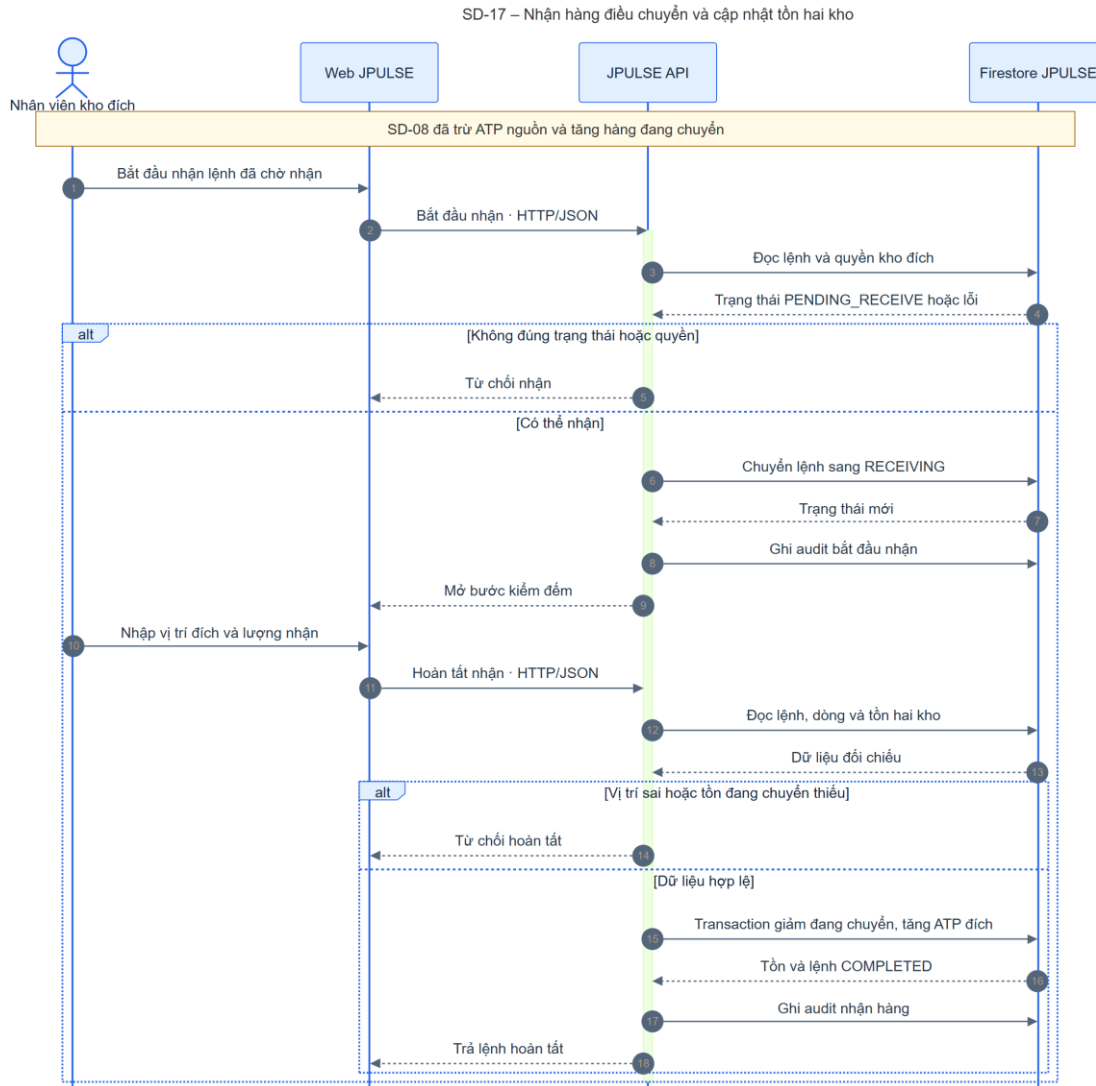
Phiếu xuất tuân theo cùng ranh giới Web → API → Firestore. Sau khi SD-06A kiểm tra chứng từ, quyền, vị trí và OTP, Hình 4-7 mô tả bước ghi phiếu và khởi tạo duyệt. Một số bước audit hoặc hoàn tất khởi tạo duyệt chạy sau batch; nếu chúng lỗi, dữ liệu phiếu có thể đã tồn tại. Luồng xuất tiếp tục ở các sơ đồ duyệt, soạn hàng và chốt phiếu SD-07–09 (SRS: BC01–BC05).

Hình 4-7: SD-06B ghi phiếu xuất và khởi tạo duyệt



Điều chuyển liên kho cần giữ quan hệ giữa phiếu xuất ở kho nguồn và việc nhận ở kho đích. Hình 4-8 mô tả bước nhận: kho đích ghi số lượng thực nhận; transaction giảm lượng đang chuyển ở nguồn, tăng ATP tại đích và lưu chênh lệch từng dòng. Chênh lệch nhận trong luồng này không tự tạo báo cáo không phù hợp; quy trình xử lý chênh lệch cần được theo dõi riêng (SRS: BC04–BC05).

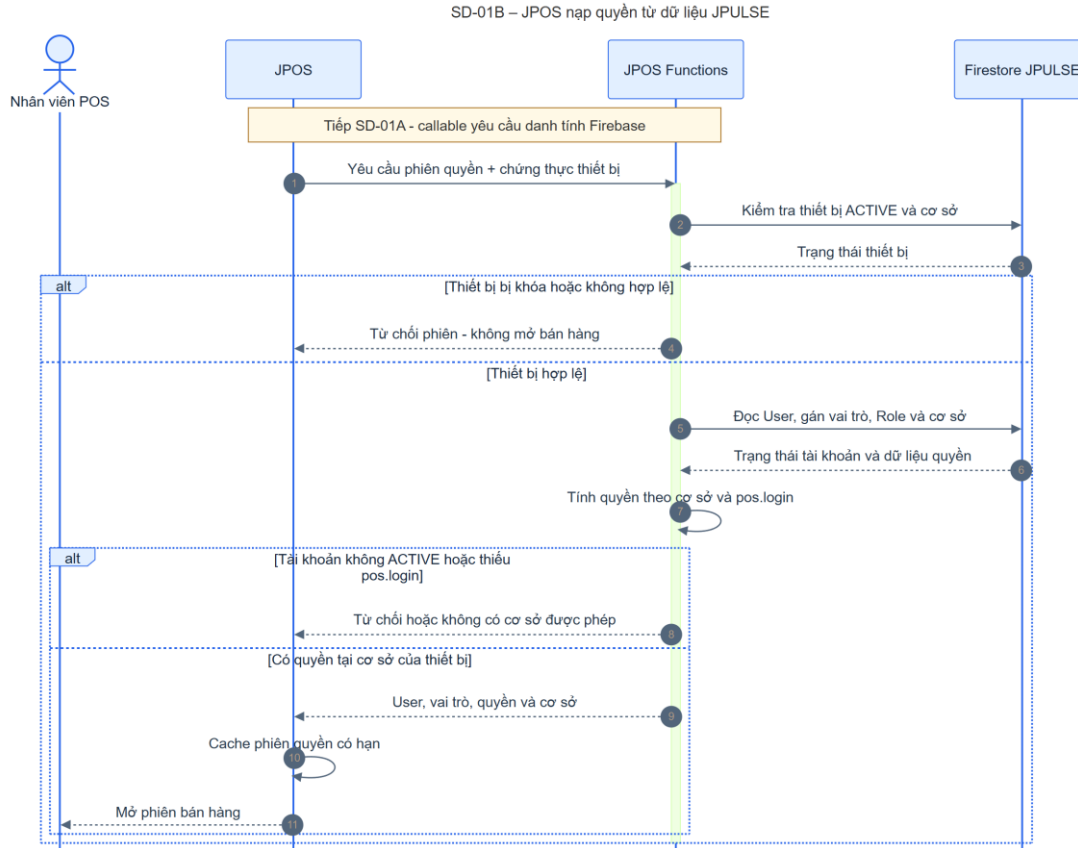
Hình 4-8: SD-17 nhận hàng điều chuyển



4.4.3. Luồng tích hợp JPOS và hóa đơn điện tử

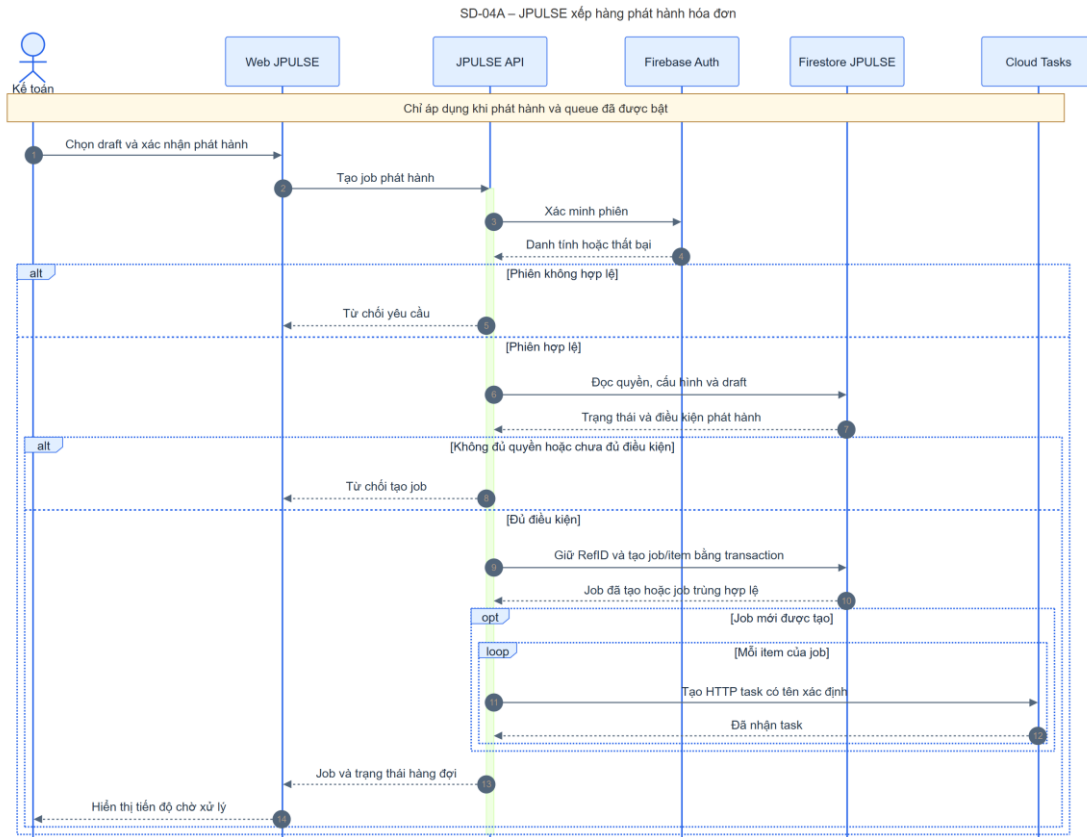
Hai loại tương tác cần được phân biệt. Hình 4-9 mô tả hiện trạng JPOS nạp quyền: sau khi Firebase xác thực, JPOS Functions đọc tài khoản, vai trò và phạm vi cơ sở từ Firestore dùng chung rồi tính quyền pos.login. Cách này khác mô tả mục tiêu trong SRS/Container Diagram rằng JPOS gọi JPULSE API để xác thực. Hình ghi lại hành vi đã được đối chiếu với mã JPOS; hợp đồng tích hợp API cho xác thực hoặc đồng bộ cần được xác minh trước khi coi hai mô tả là một. Các sơ đồ SD-01A/C lần lượt nêu bước xác thực ban đầu và giới hạn cache khi mất mạng.

Hình 4-9: SD-01B JPOS nạp quyền từ dữ liệu JPULSE



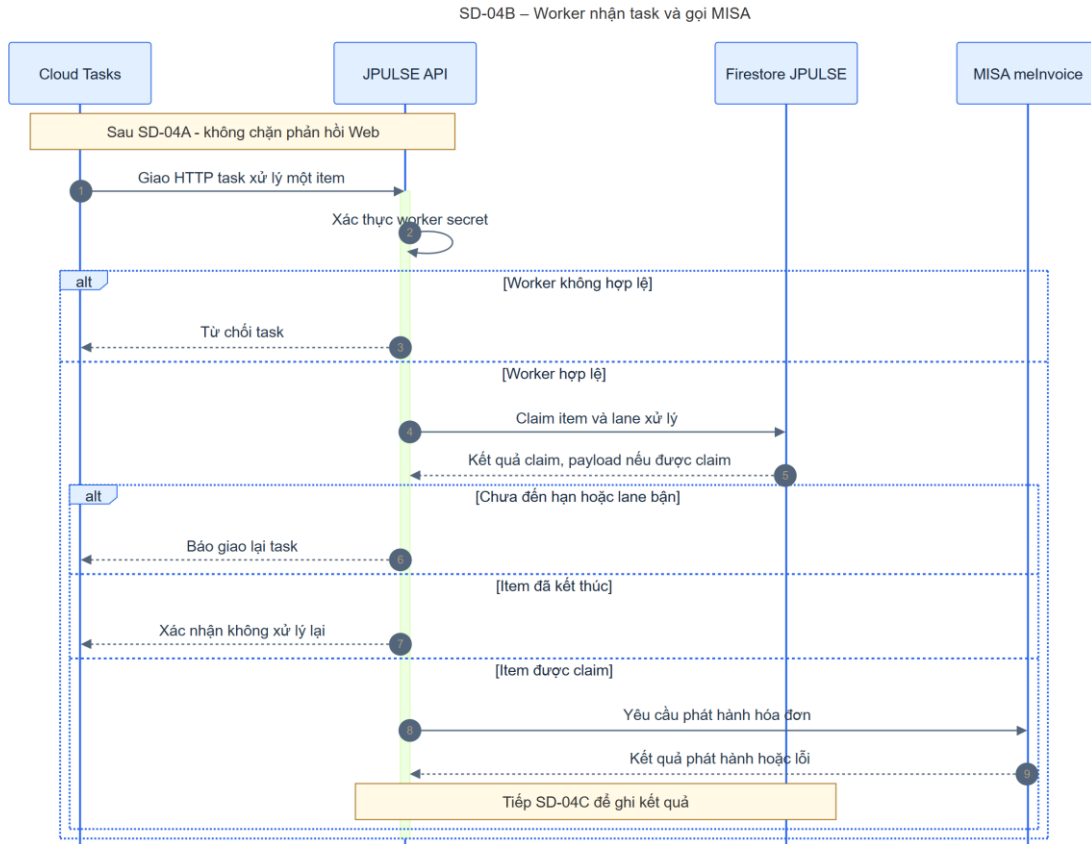
Với hóa đơn điện tử, Hình 4-10 cho thấy API kiểm tra phiên, quyền, cấu hình và bản nháp trước khi ghi job/item và tạo Cloud Task. Web nhận mã công việc và trạng thái hàng đợi mà không chờ MISA. Khóa tham chiếu và tên task ổn định giúp ngăn tạo trùng khi yêu cầu được gửi lại (SRS: P04, INT-03, INT-10).

Hình 4-10: SD-04A tiếp nhận và xếp hàng phát hành hóa đơn



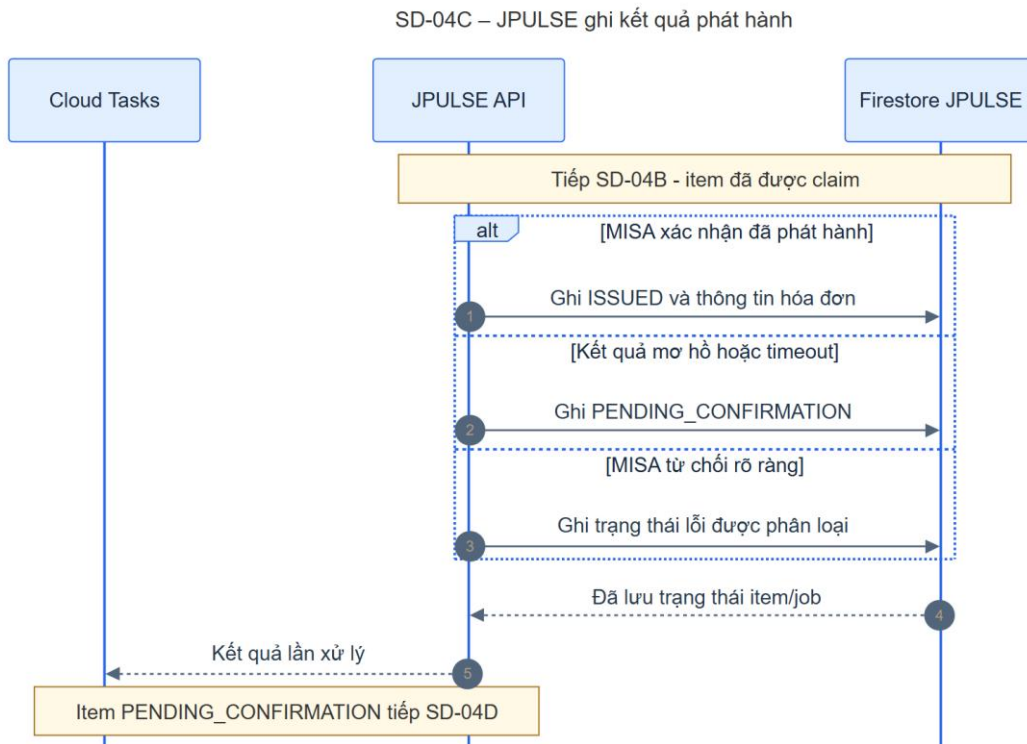
Hình 4-11 mô tả Cloud Tasks gọi worker của JPULSE API. Worker xác thực lời gọi, claim item/lane trong Firestore và chỉ gửi yêu cầu phát hành tới MISA khi claim thành công. Task đến sớm, lane bận hoặc item đã kết thúc được xử lý theo nhánh riêng, tránh phát hành lại ngoài ý muốn.

Hình 4-11: SD-04B worker nhận task và gọi MISA



Hình 4-12 thể hiện bước ghi kết quả MISA: thành công thành ISSUED; timeout hoặc phản hồi mơ hồ thành PENDING_CONFIRMATION; lỗi xác định được phân loại. Kết quả chưa rõ không được tự phát hành lại ngay mà phải qua bước xác nhận/đối soát ở SD-04D. Cách lưu trạng thái này cho phép người dùng theo dõi và xử lý thủ công mà không nhân đôi hóa đơn (SRS: BC06–BC07, AVL04–AVL08).

Hình 4-12: SD-04C ghi kết quả phát hành hóa đơn



4.5. Góc nhìn triển khai

Ở mức triển khai, SRS xác định backend Node.js/Express được đóng gói Docker và định hướng chạy trên Google Cloud Run tại asia-southeast1; Web Next.js được build ở chế độ standalone. Firestore, Cloud Storage và Firebase Authentication là dịch vụ được quản lý; Cloud Tasks tham gia các luồng nền khi queue, URL worker và quyền gọi được cấu hình. Các thành phần phải dùng cấu hình, secret và service account theo môi trường; API/worker cần xác thực lời gọi, ghi log và health check. Trạng thái nghiệp vụ nằm trong kho bền vững để instance Cloud Run có thể thay đổi hoặc mở rộng (SRS: TC01–TC06, TC08, TC11–TC13, SCA01–SCA02).

Kho biểu đồ chưa có Deployment Diagram xác định nơi chạy Web, ingress, IAM, queue/worker và vị trí Firestore/Storage theo môi trường. Vì vậy Mục 4.5 chỉ ghi các ràng buộc đã được SRS xác nhận, chưa khẳng định topology production. Cần bổ sung hình sau khi chốt môi trường thực tế và đối chiếu kế hoạch vận hành ở Chương 6.

Chương 5: Dữ liệu, giao tiếp và luồng xử lý

5.1. Tổng quan dữ liệu của hệ thống

5.2. Cấu trúc dữ liệu và nơi lưu trữ

5.2.1. Dữ liệu nghiệp vụ và phân quyền

5.2.2. Tập và tài liệu

5.2.3. Dữ liệu tích hợp và nhật ký hệ thống

Bảng 5-1: Nhóm dữ liệu và nơi lưu trữ

Nhóm dữ liệu	Nơi lưu trữ	Ghi chú

5.3. Luồng dữ liệu giữa các thành phần

5.4. Giao diện kết nối và API chính

5.4.1. API giữa giao diện web và máy chủ

5.4.2. Kết nối với dịch vụ bên ngoài

5.5. Xác thực, phân quyền và bảo vệ dữ liệu

5.6. Các luồng xử lý chính

5.6.1. Xác thực và truy cập theo vai trò, cơ sở

5.6.2. Quản lý nhập xuất, điều chuyển và kiểm kê

5.6.3. Ghi nhận doanh thu và phát hành hóa đơn

5.6.4. Chấm công và nghỉ phép

5.7. Xử lý lỗi và tính nhất quán dữ liệu

Chương 6: Triển khai và vận hành

6.1. Các môi trường triển khai

6.1.1. Môi trường phát triển

6.1.2. Môi trường kiểm thử

6.1.3. Môi trường vận hành

6.2. Sơ đồ triển khai

Hình 6-1: Sơ đồ triển khai JPULSE

6.3. Cấu hình ứng dụng và quản lý thông tin bí mật

6.4. Quy trình triển khai và cập nhật

6.5. Ghi log và giám sát

6.6. Sao lưu và khôi phục dữ liệu

6.7. Khả năng mở rộng và giới hạn vận hành

Chương 7: Quyết định kiến trúc và đánh giá

7.1. Danh sách quyết định kiến trúc quan trọng

Bảng 7-1: Danh sách quyết định kiến trúc

Mã	Bối cảnh	Quyết định	Hệ quả

7.2. Lý do lựa chọn công nghệ và phương án thiết kế

7.3. Các phương án đã cân nhắc

7.4. Đánh đổi của phương án được chọn

7.5. Đánh giá kiến trúc theo yêu cầu chất lượng

7.6. Rủi ro kỹ thuật và hướng xử lý

Chương 8: Kết luận và phụ lục

8.1. Tóm tắt kiến trúc hệ thống

8.2. Những giới hạn hiện tại

8.3. Hướng phát triển kiến trúc

8.4. Phụ lục sơ đồ

8.5. Phụ lục bảng mô tả thành phần, dữ liệu hoặc API

Bảng 8-1: Danh mục thành phần, dữ liệu hoặc API

Mã	Tên	Trách nhiệm	Tham chiếu

8.6. Danh mục thuật ngữ